

# Lecture 12:

# **Neural Networks**

October 5, 2026

we now know how to sample high dimension distributions:  
*rejection sampling, importance sampling, MCMC, Hamiltonian MC*

let's apply it to astro! choose your adventure:

**ast384c/inclass/lec9\_hierarchical/review\_emission.ipynb**

**ast384c/inclass/lec9\_hierarchical/review\_sed.ipynb**

**in-class: population inference using hierarchical Bayes**

**ast384c/inclass/lec11\_gp/gp\_lightcurve.ipynb**

# Neural Networks

gaussian processes is not scalable to large datasets

$\{f(x_1) \dots f(x_n)\} \sim \mathcal{N}(0, K)$  where  $K$  is an  $n \times n$  matrix

gaussian processes is not scalable to large datasets

$\{f(x_1) \dots f(x_n)\} \sim \mathcal{N}(0, K)$  where  $K$  is an  $n \times n$  matrix

if  $n = 16,020$  then  $K$  would take up 2.1GB plus inverting it  
 $n^2$  in memory and  $n^3$  in compute time

gaussian processes = Bayesian linear regression on **fixed** basis functions

$$f(x) = \sum_m w_m \phi_m(x)$$

gaussian processes = Bayesian linear regression on **fixed** basis functions

$$f(x) = \sum_m w_m \phi_m(x)$$

neural network = basis expansion with a **learned** basis

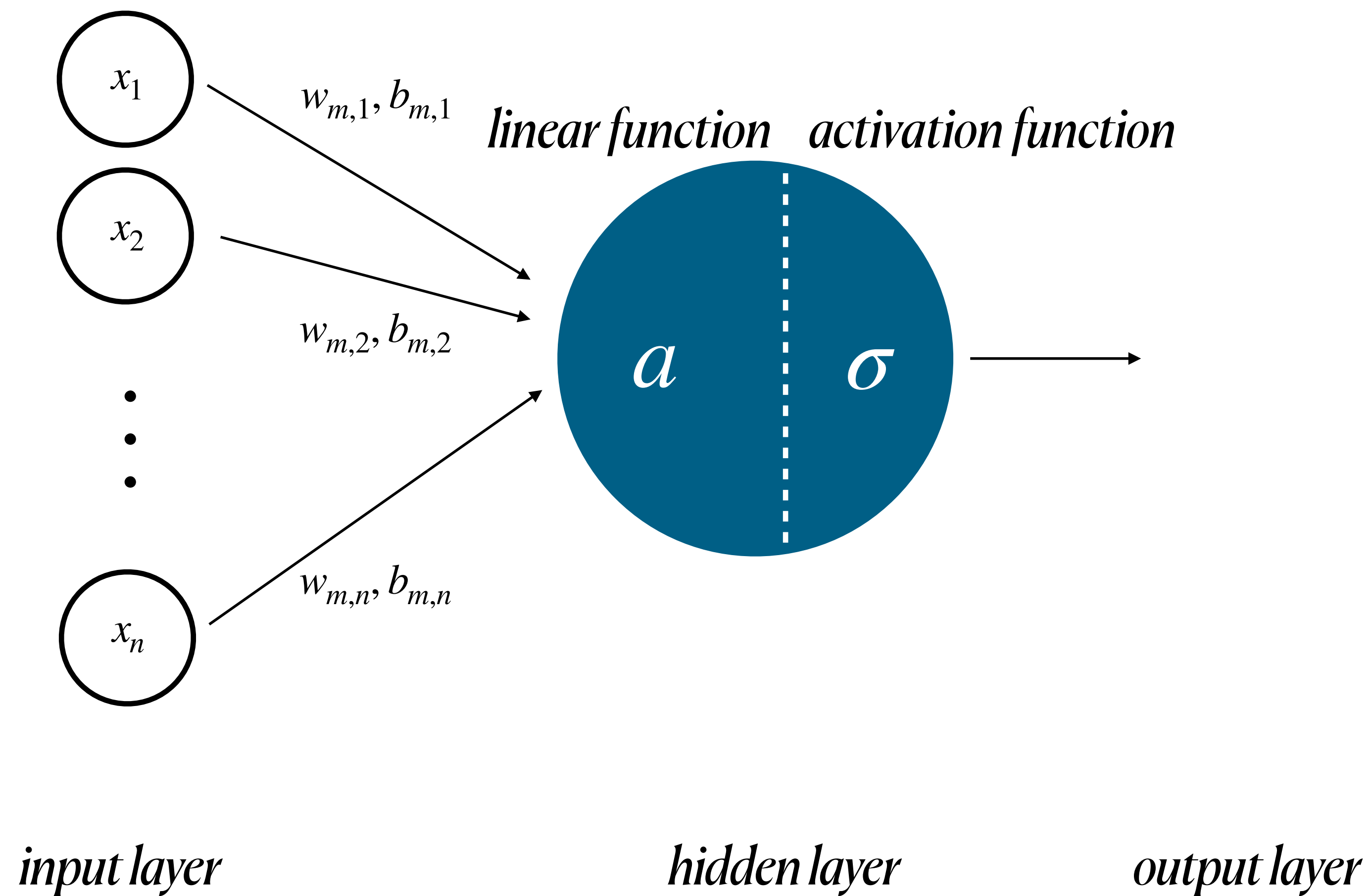
$$f(x) = \sum_m v_m \sigma(w_m^T x + b_m)$$

*neural network with one hidden layer*

$$f(x) = \sum_m v_m \sigma(w_m^T x + b_m)$$

$$f(x) = \sum_m v_m \sigma(w_m^T x + b_m)$$

**single neuron**



**activation function ( $\sigma$ ) — introduces non-linearities**

$$f(x) = \sum_m v_m \sigma(w_m^T x + b_m)$$

**activation function ( $\sigma$ ) — introduces non-linearities**

$$f(x) = \sum_m v_m \sigma(w_m^T x + b_m)$$



with no activation function

$$f(x) = \sum_m v_m (w_m^T x + b_m)$$

neural network is linear

**activation function ( $\sigma$ )** — introduces non-linearities

ReLU (*Rect-Linear Unit*)  $\sigma(a) = \max(0, a)$

tanh  $\sigma(a) = \tanh(a)$

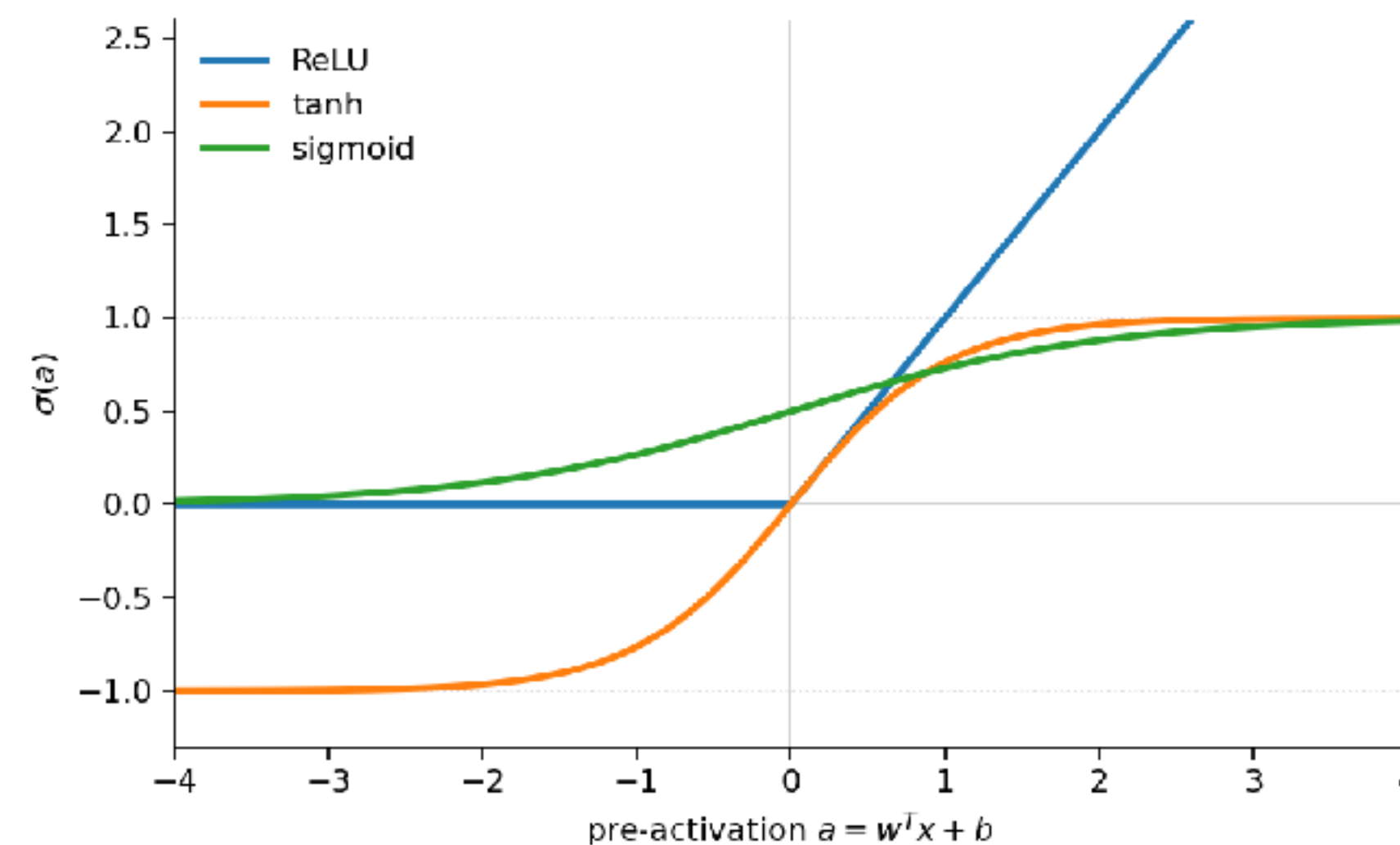
sigmoid  $\sigma(a) = 1/(1 + e^{-a})$

**activation function ( $\sigma$ )** — introduces non-linearities

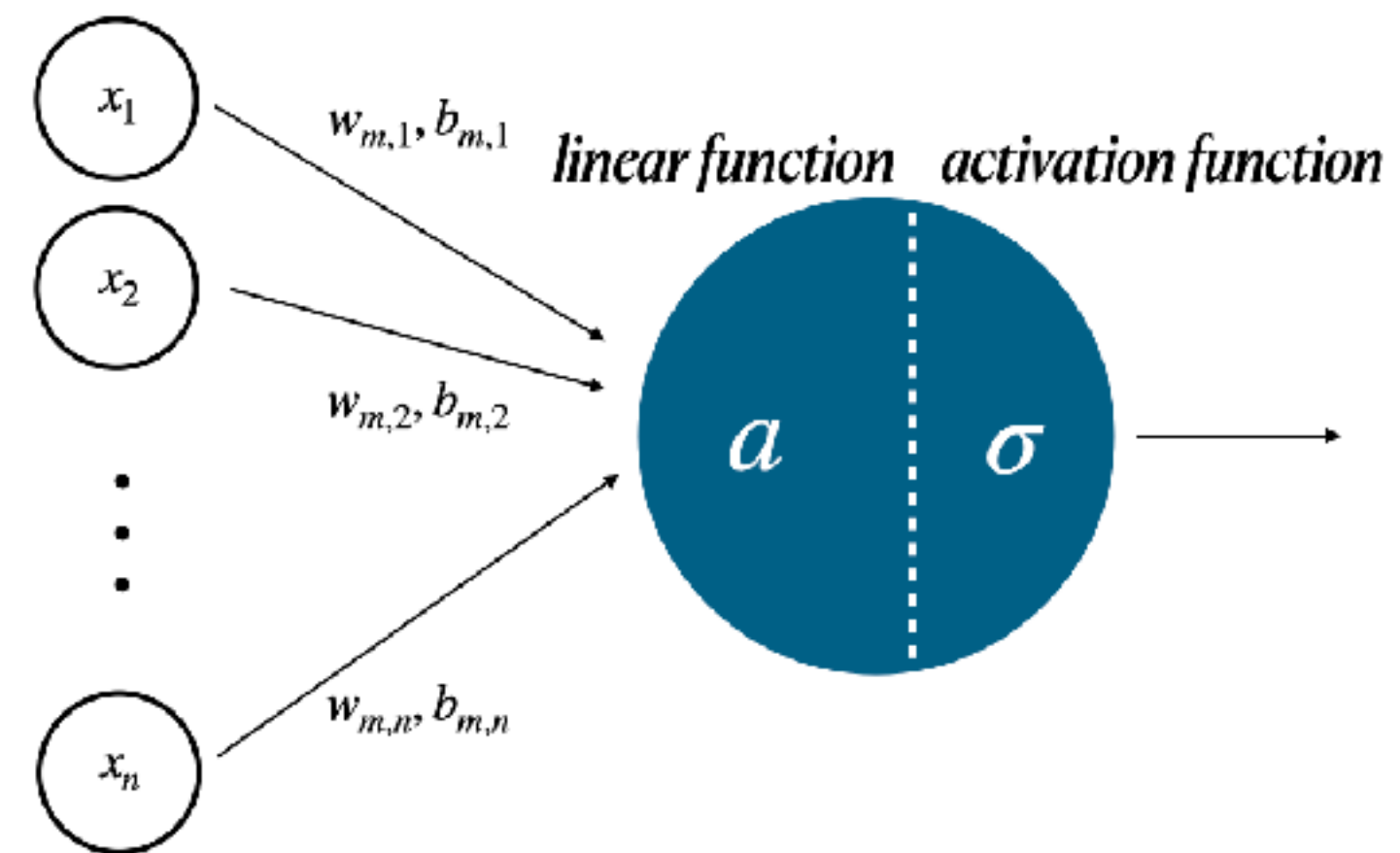
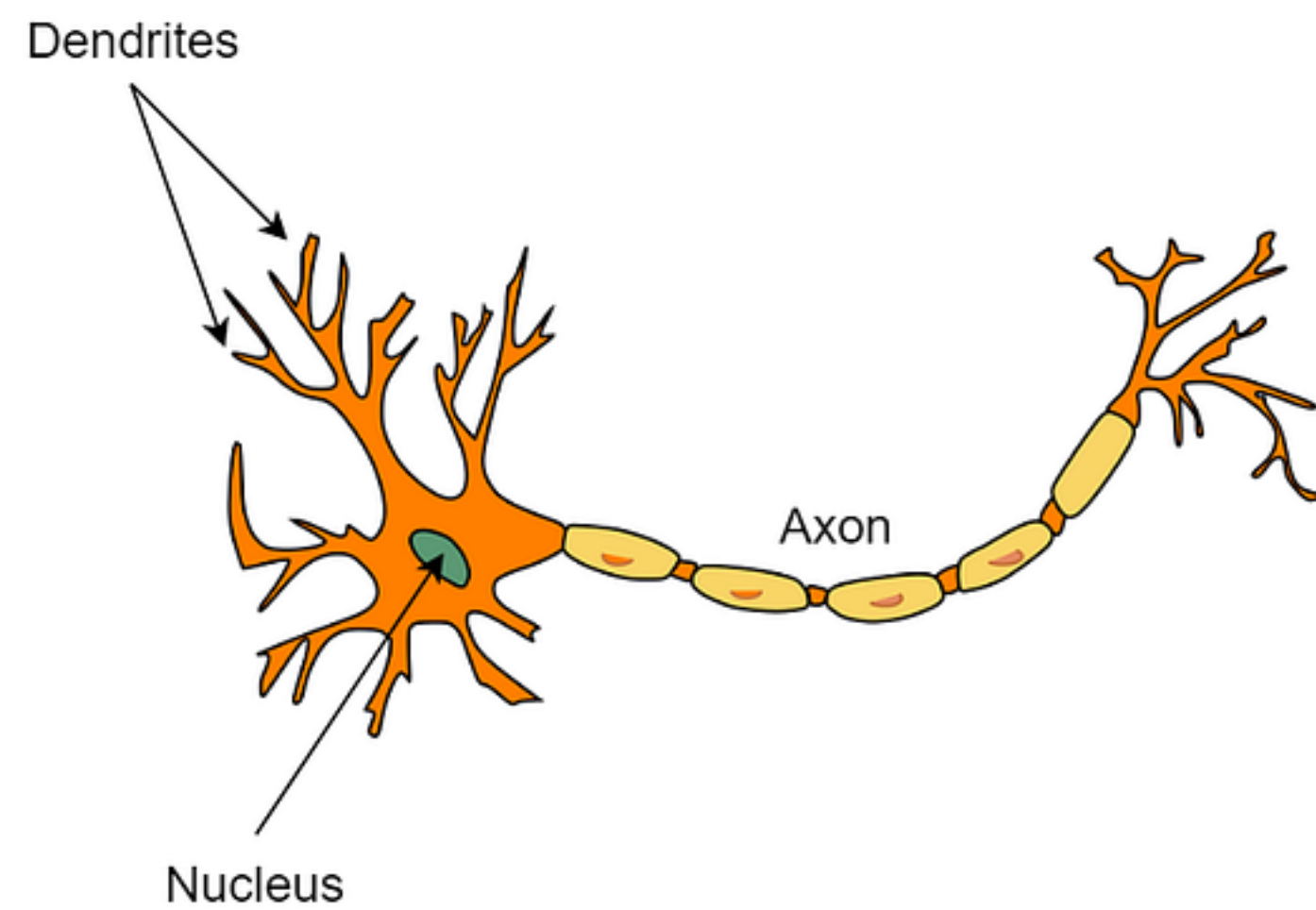
ReLU (*Rect-Linear Unit*)  $\sigma(a) = \max(0, a)$

tanh  $\sigma(a) = \tanh(a)$

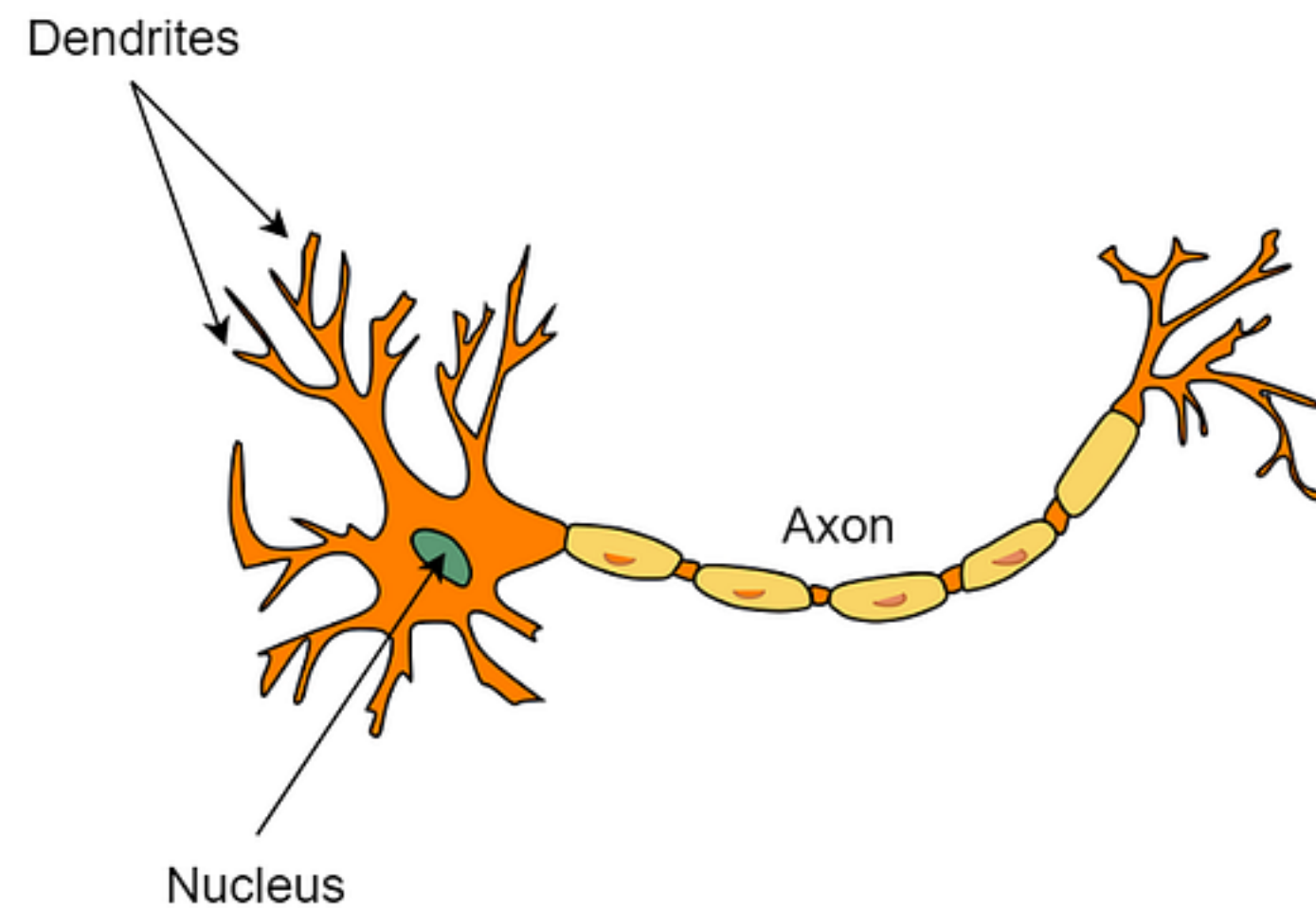
sigmoid  $\sigma(a) = 1/(1 + e^{-a})$



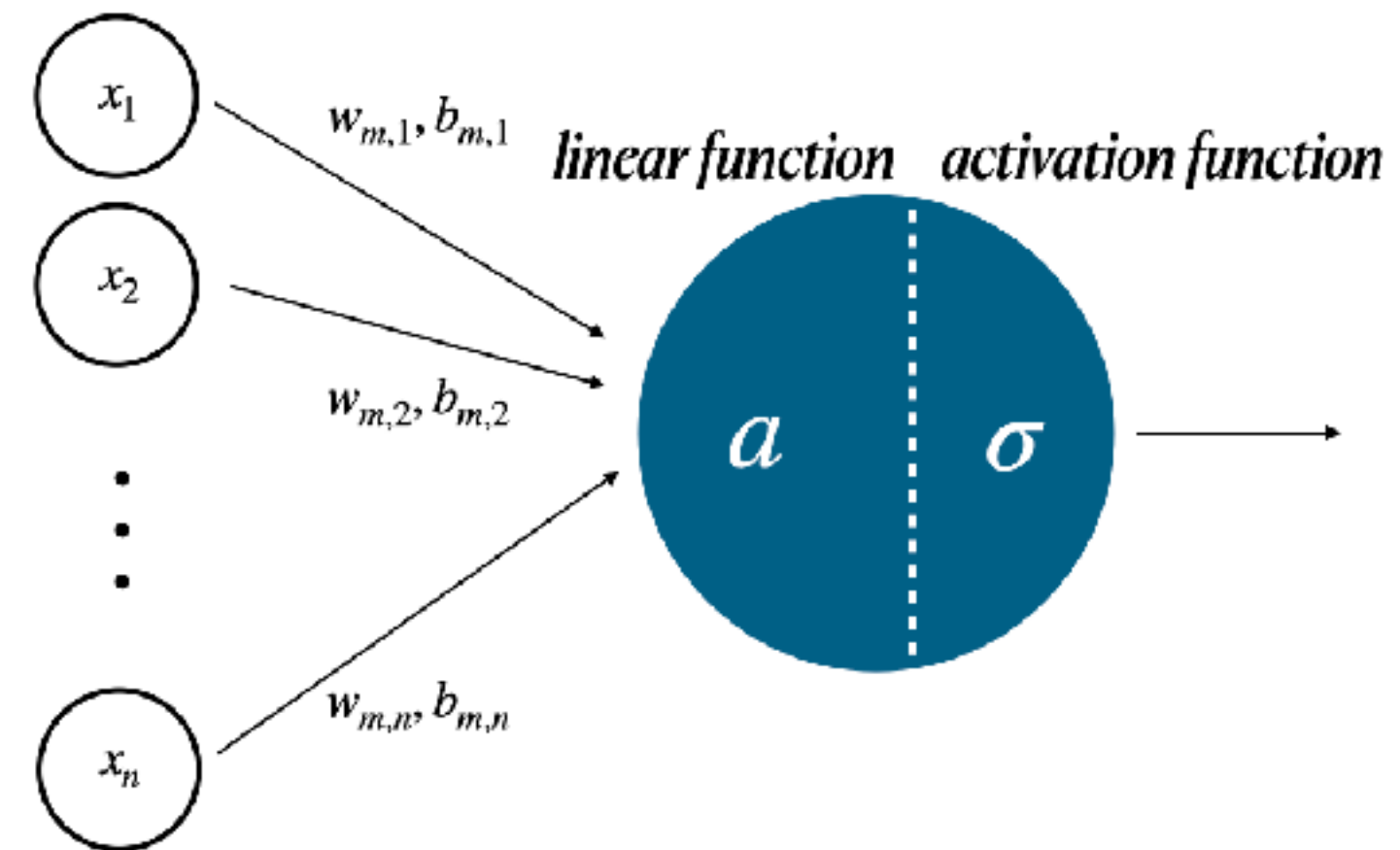
# neural networks



# neural networks

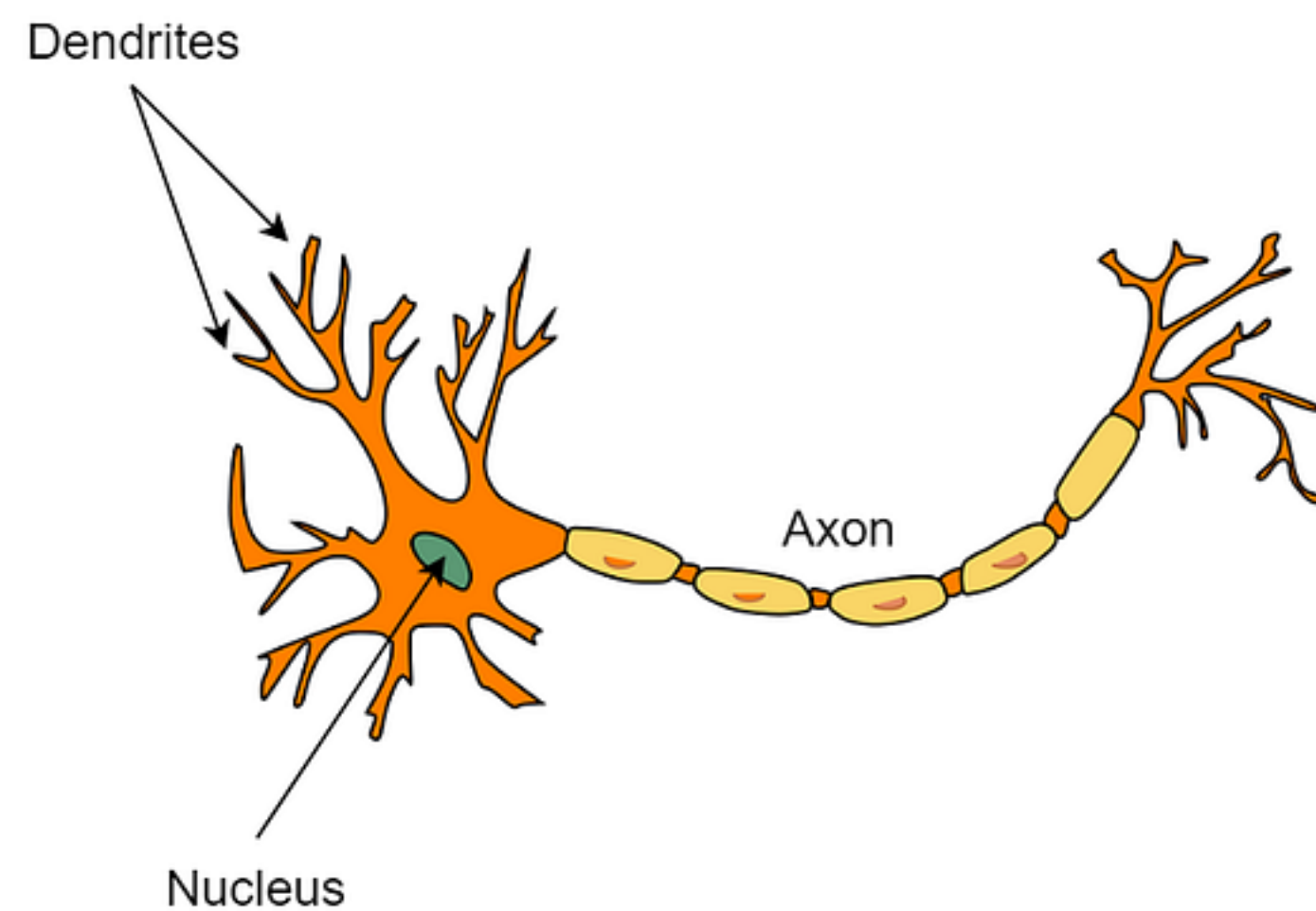


dendrites receive signals  
from other neurons

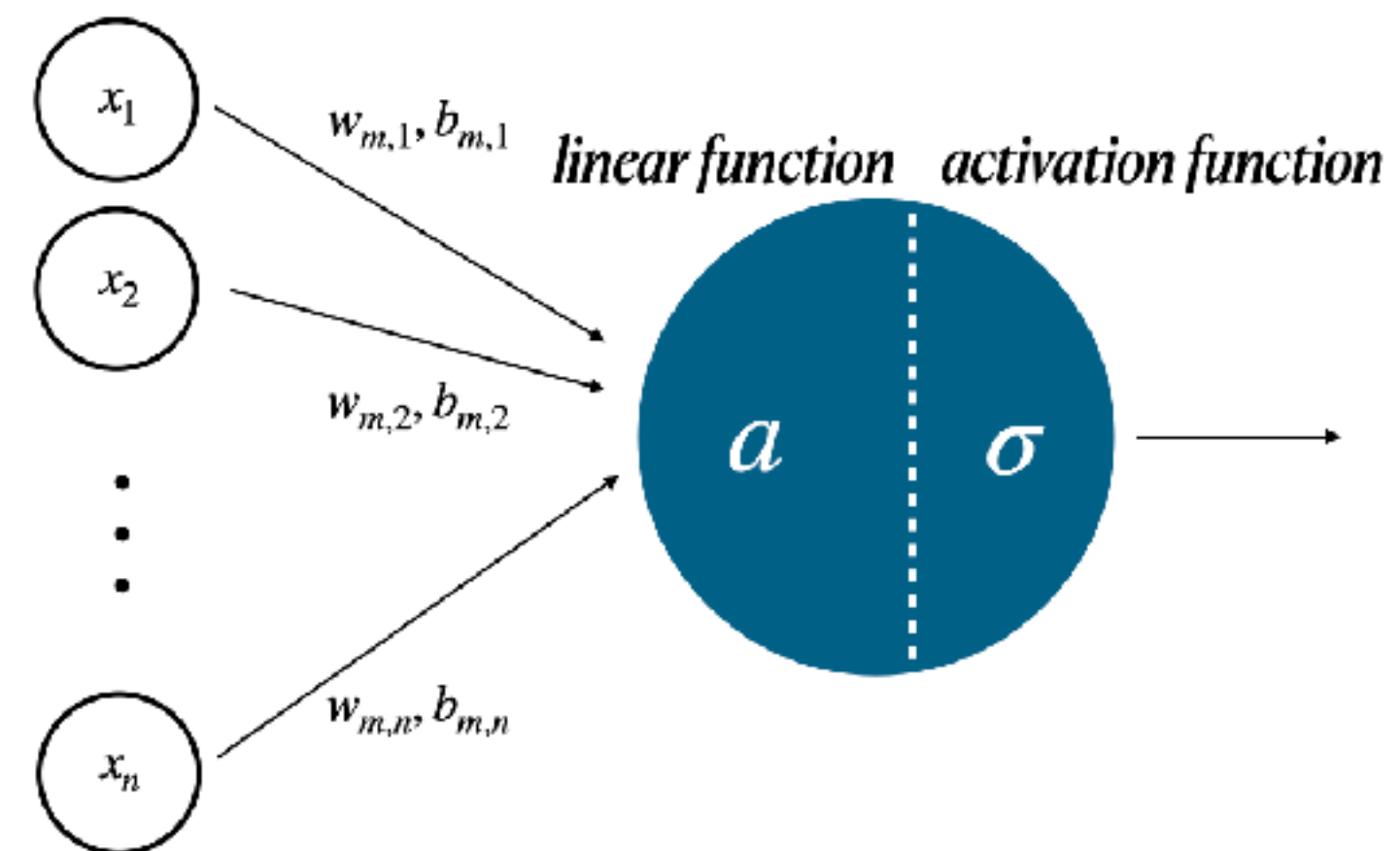


inputs  $x_1, x_2 \dots x_n$

# neural networks

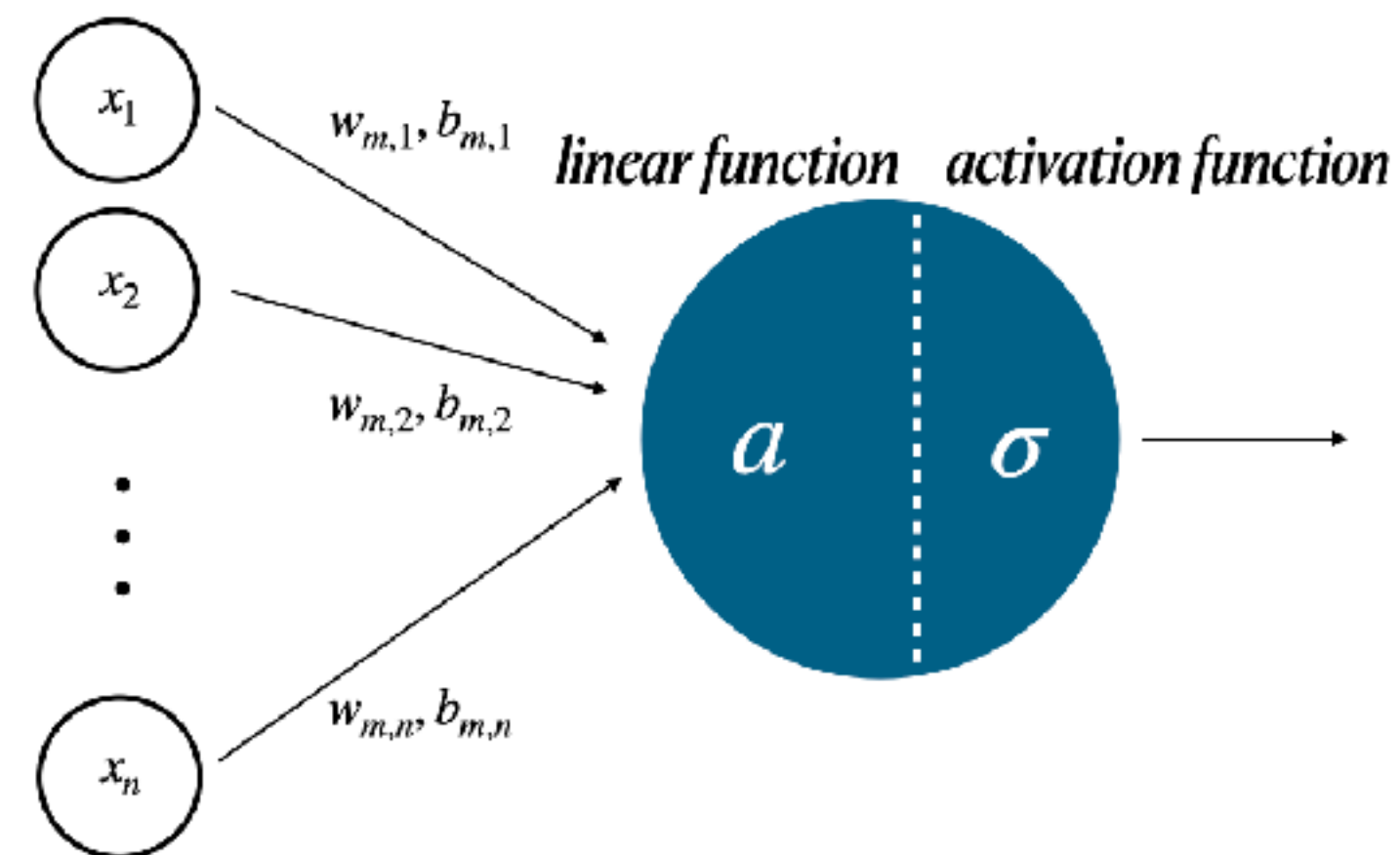
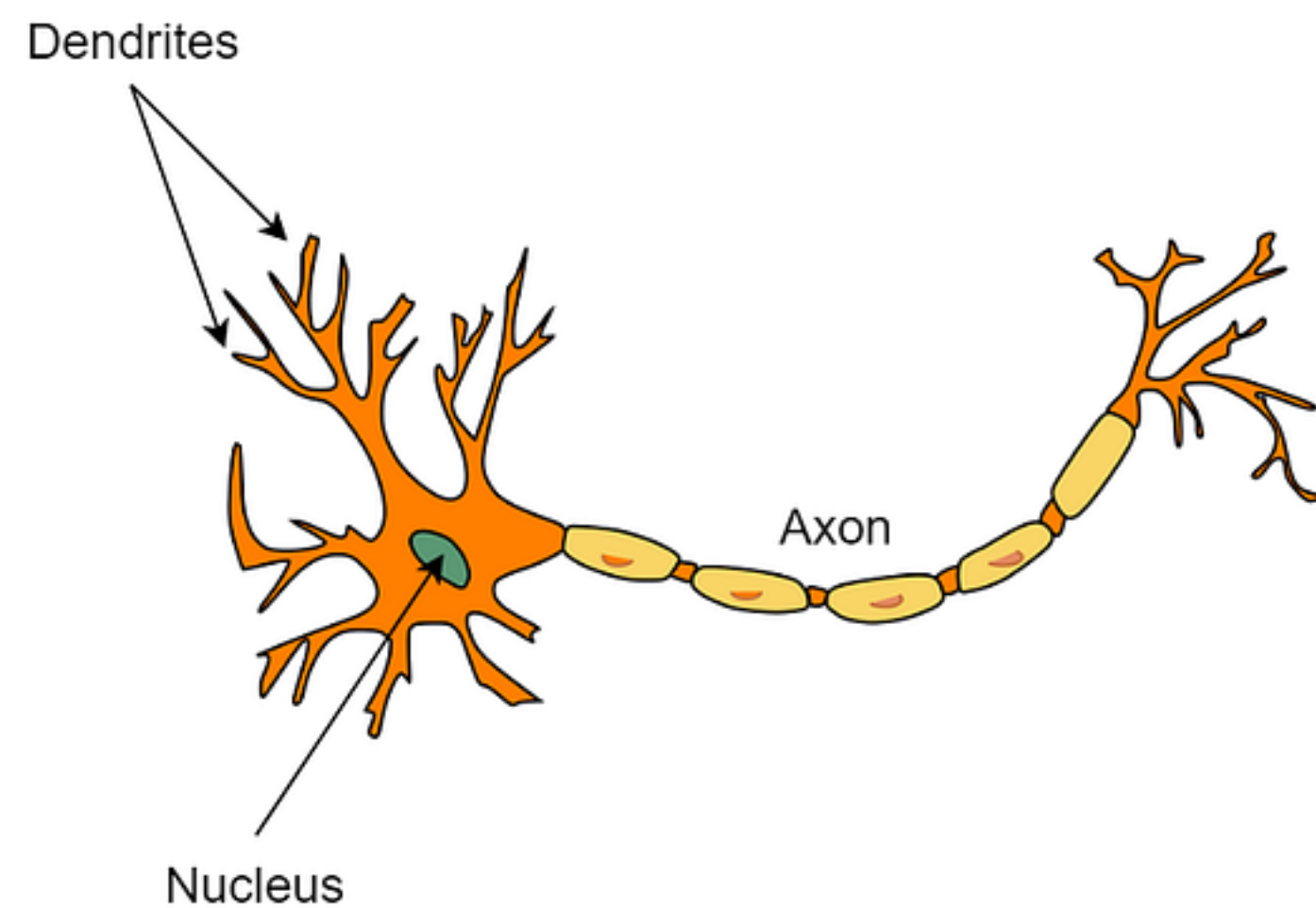


synapses pass them on



weights,  $w_i$

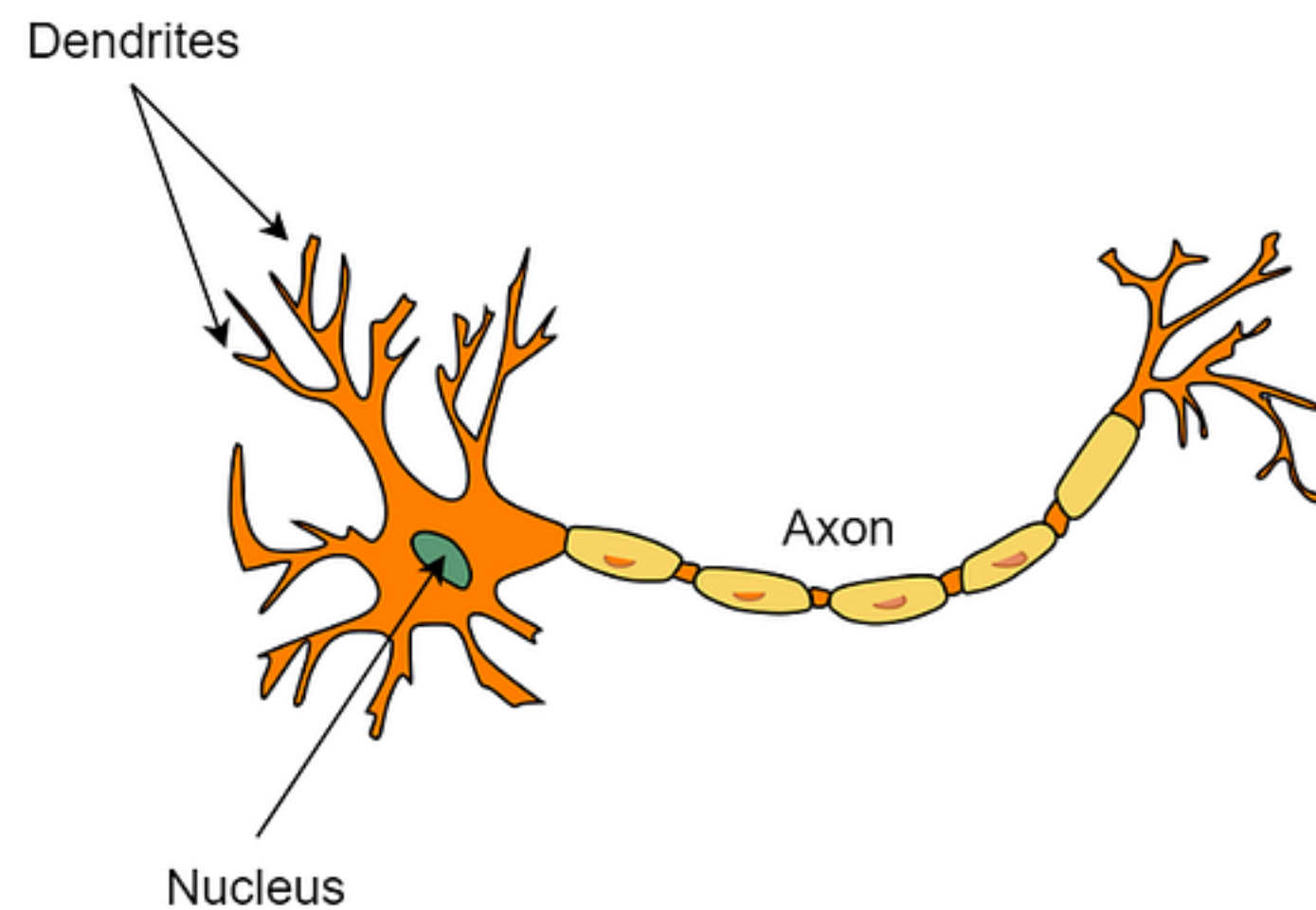
# neural networks



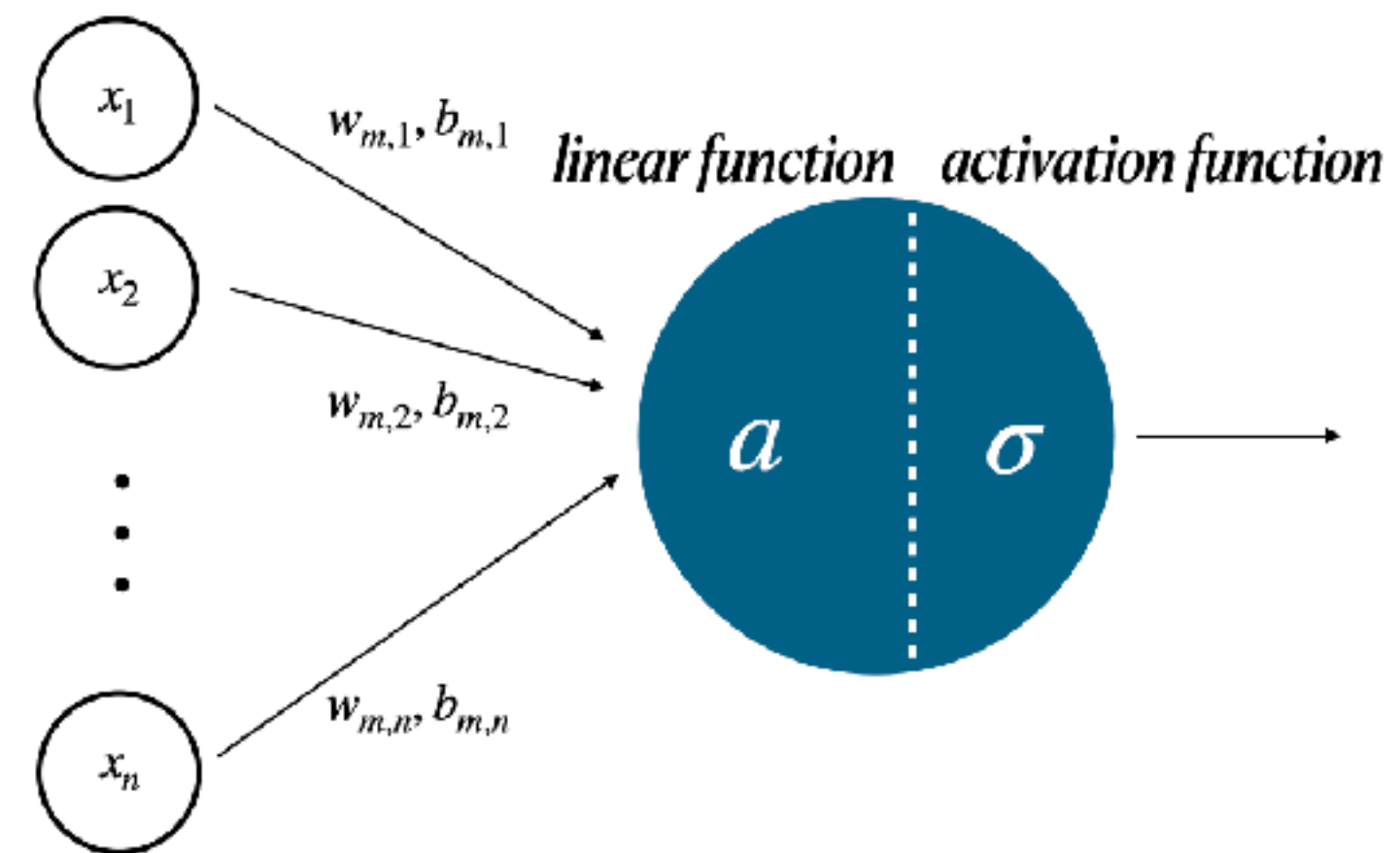
the cell body adds them up

$$\sum_i w_i x_i + b$$

# neural networks

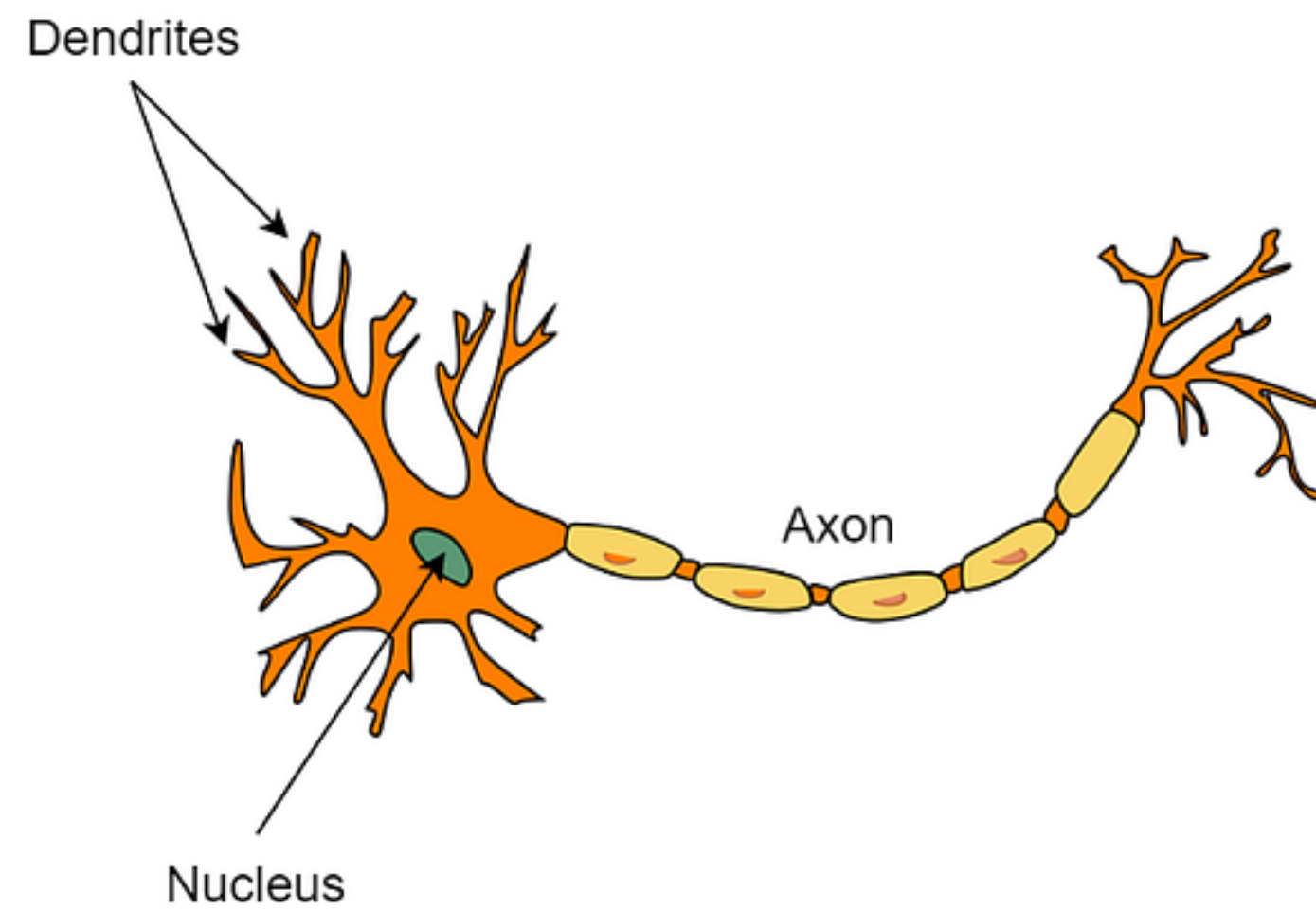


if fires if the total crosses a  
threshold

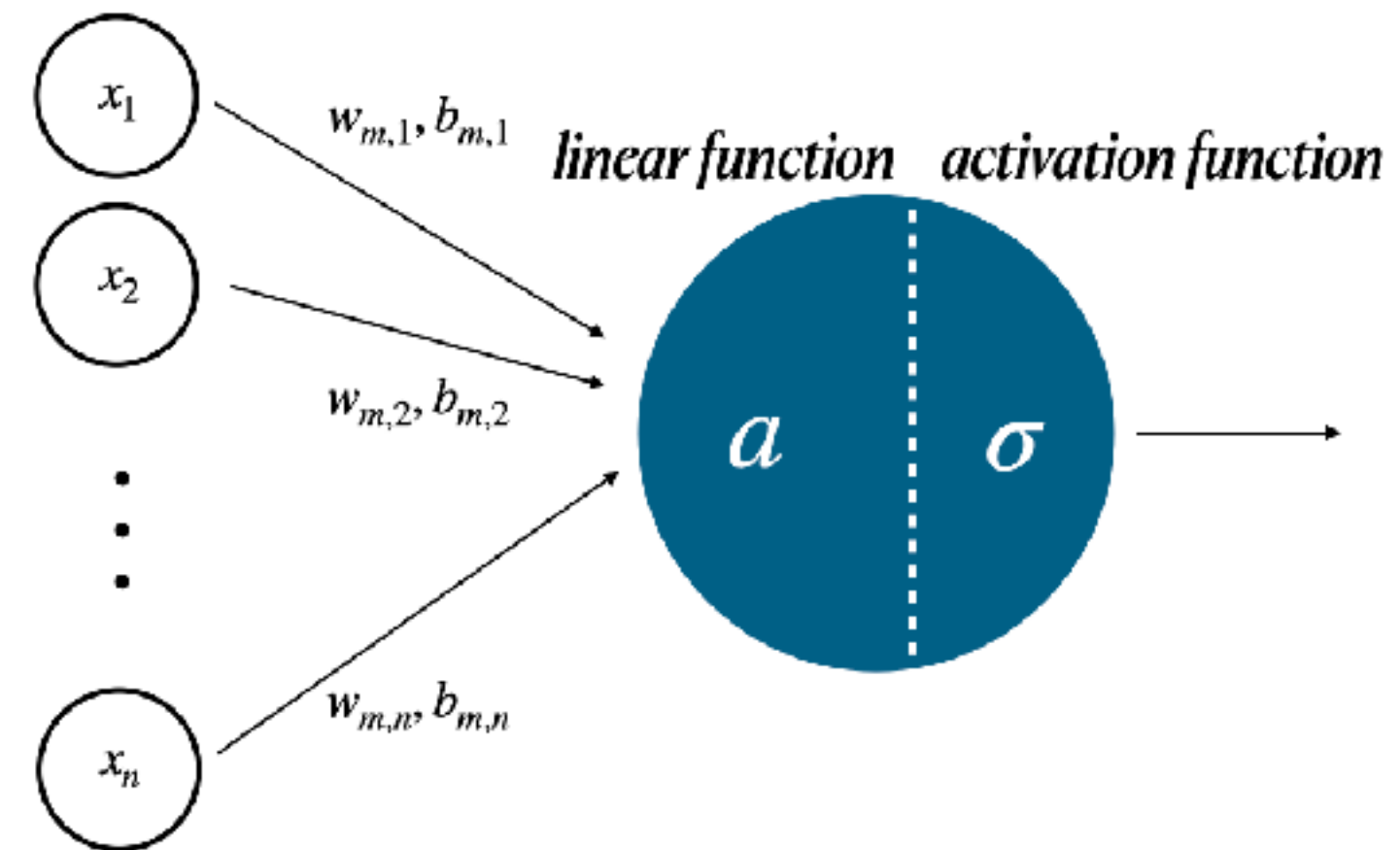


activation function  $\sigma$

# neural networks

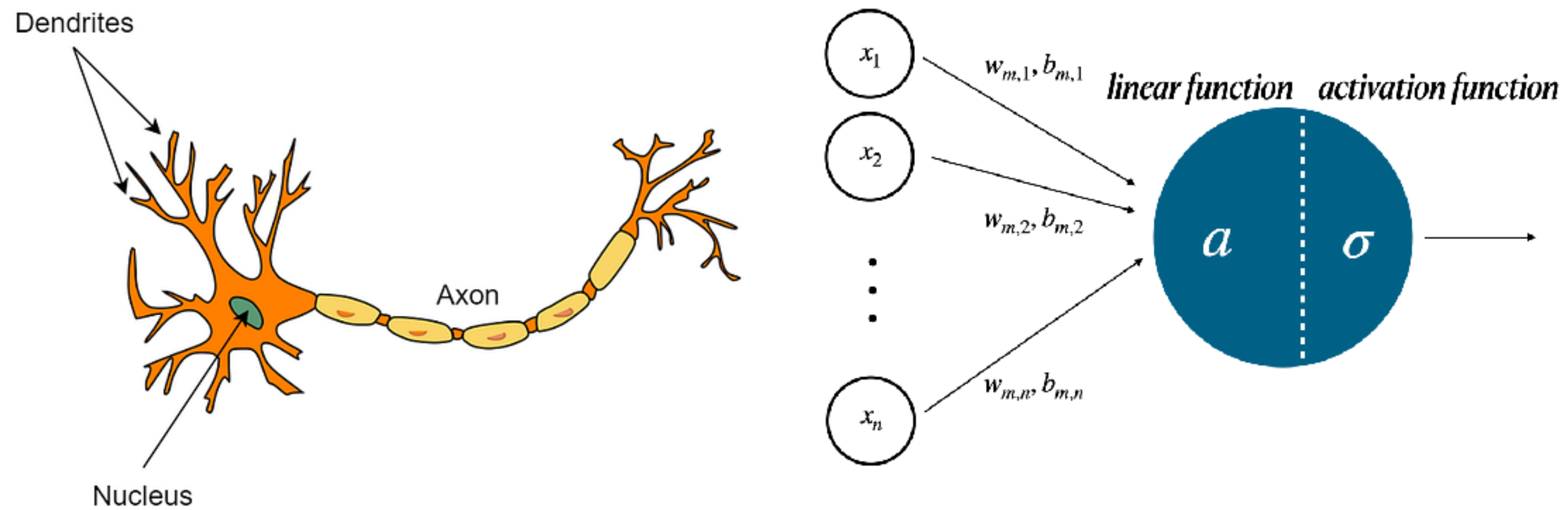


axon carries the output to  
the next cell



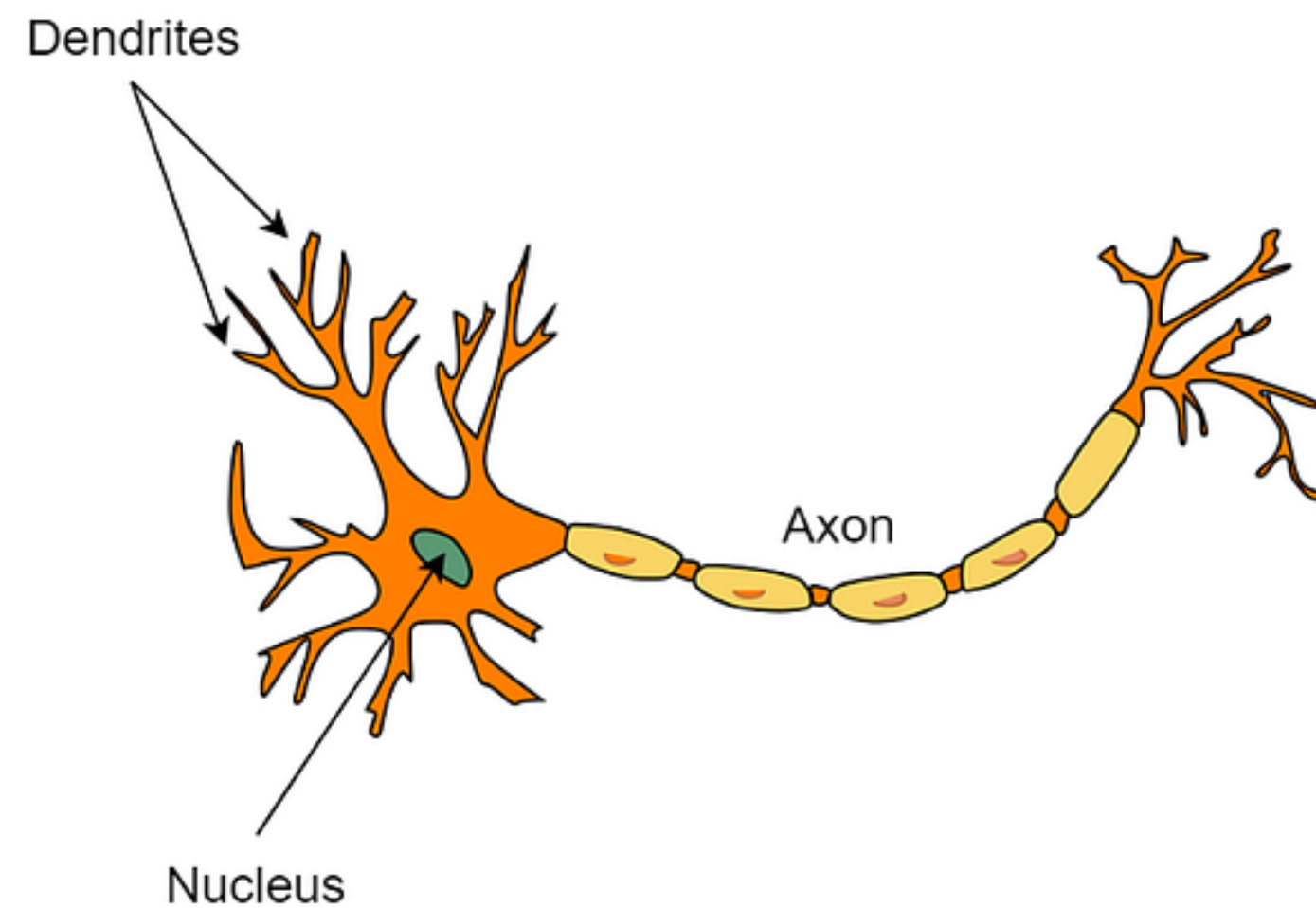
output feeds the next  
layer

# neural networks

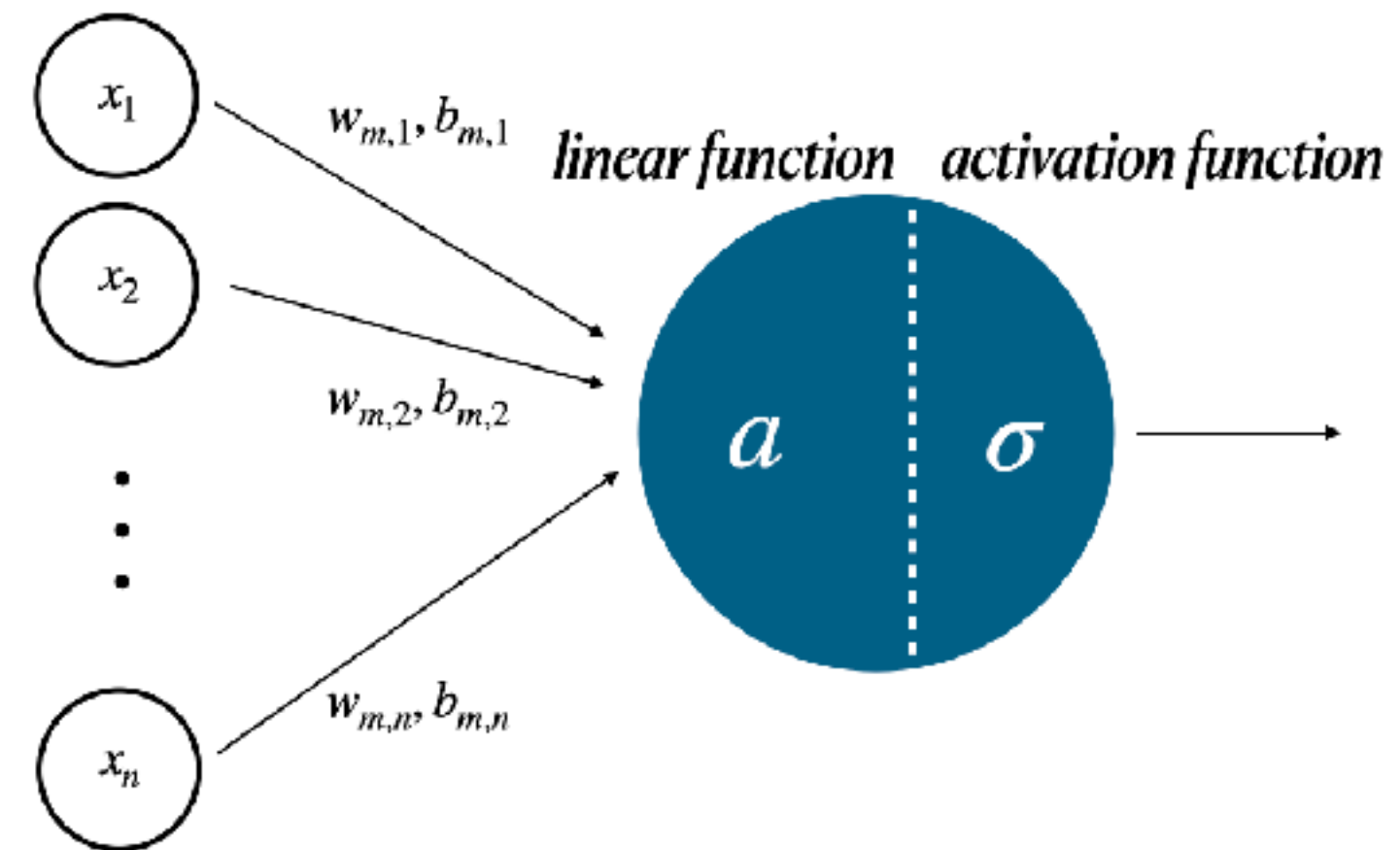


artificial neurons (*McCulloch & Pitts 1943*)

# neural networks

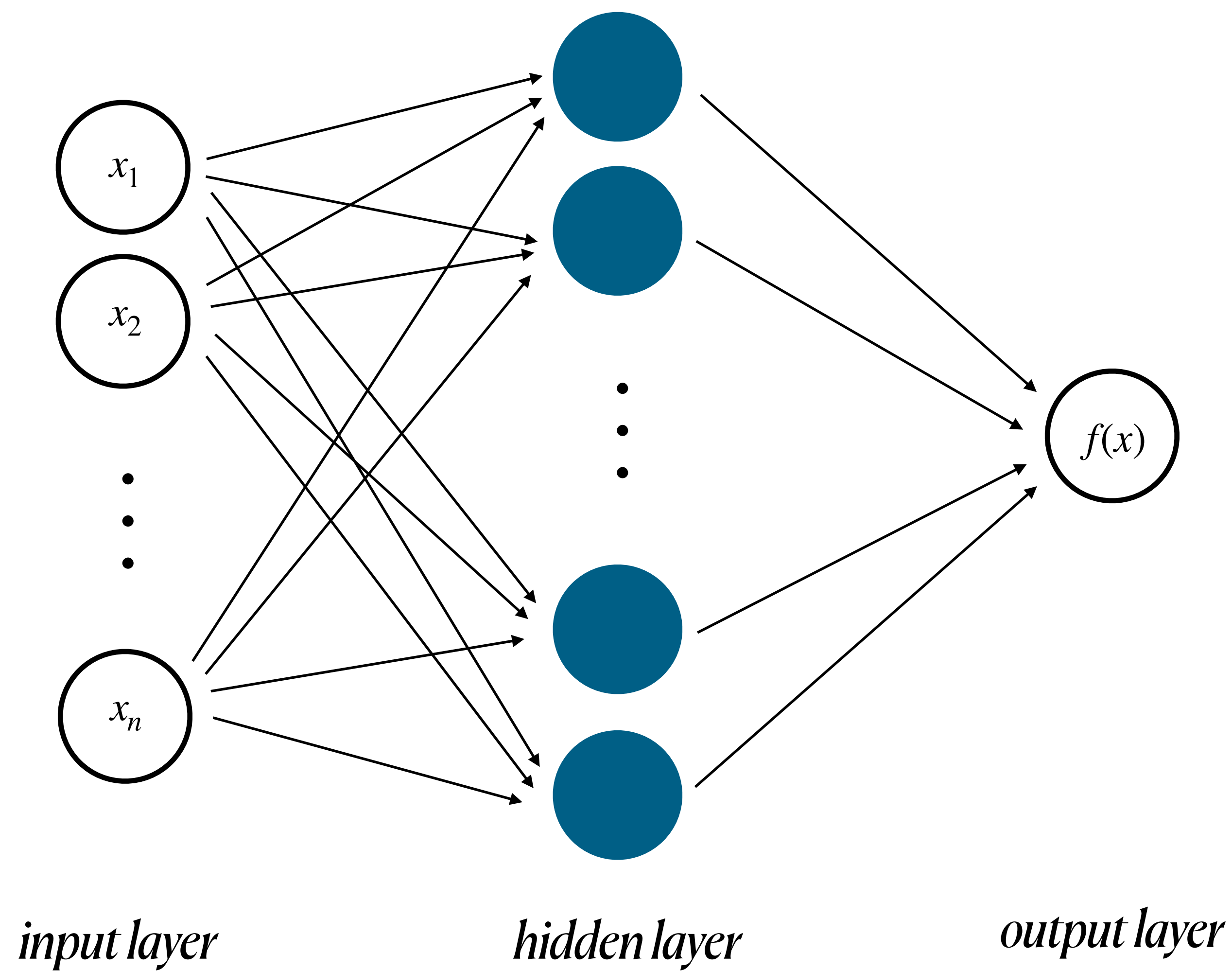


learning strengthens the  
synapses (*Hebb 1949*)



learning adjusts the  
weights (*Rosenblatt 1958*)

$$f(x) = \sum_m v_m \sigma(w_m^T x + b_m)$$



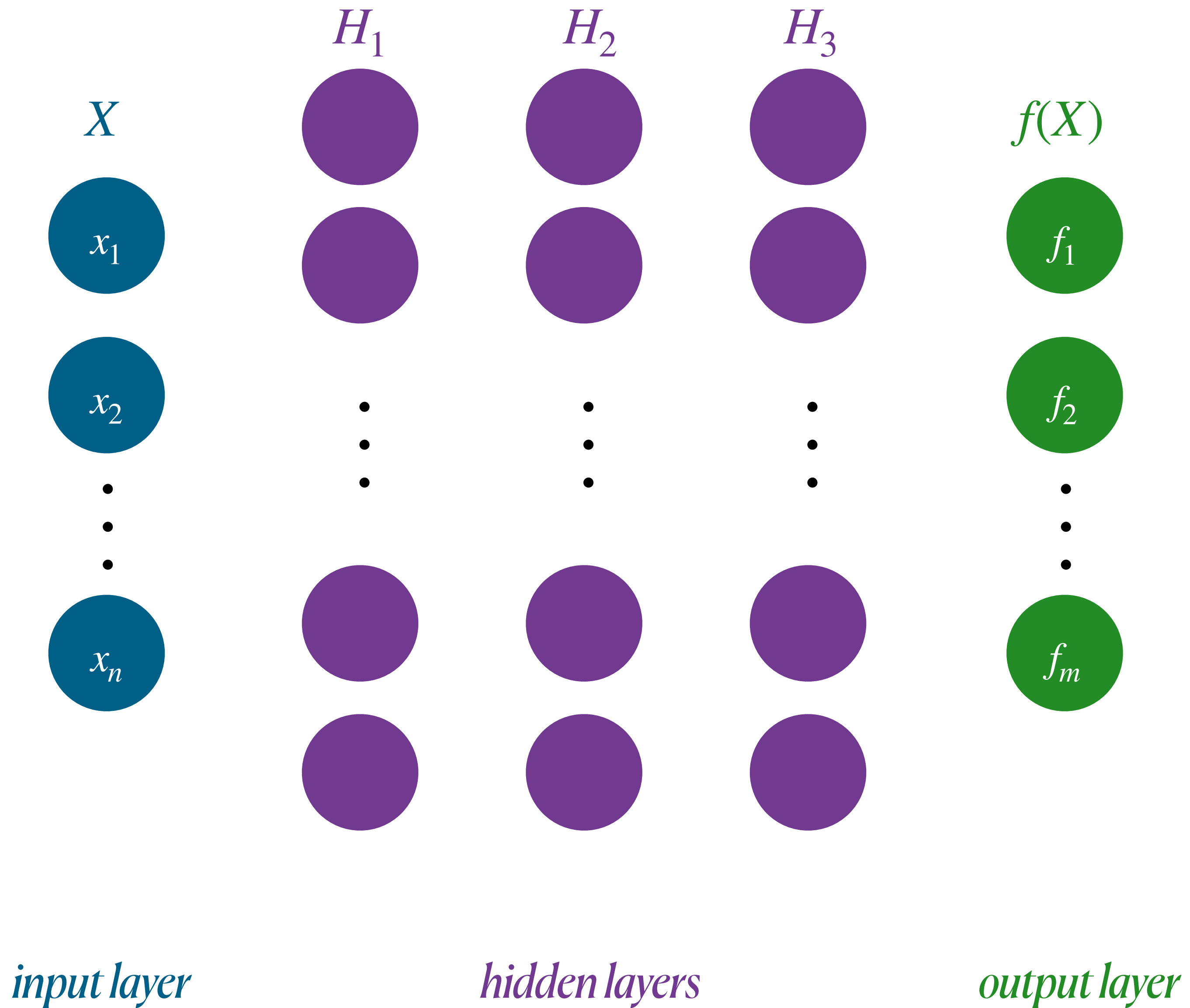
**universal approximation:** a single hidden layer with enough units can approximate any continuous function on a bounded region to any accuracy (*Cybenko 1989; Hornik 1991*)

**universal approximation:** a single hidden layer with enough units can approximate any continuous function on a bounded region to any accuracy (*Cybenko 1989; Hornik 1991*)

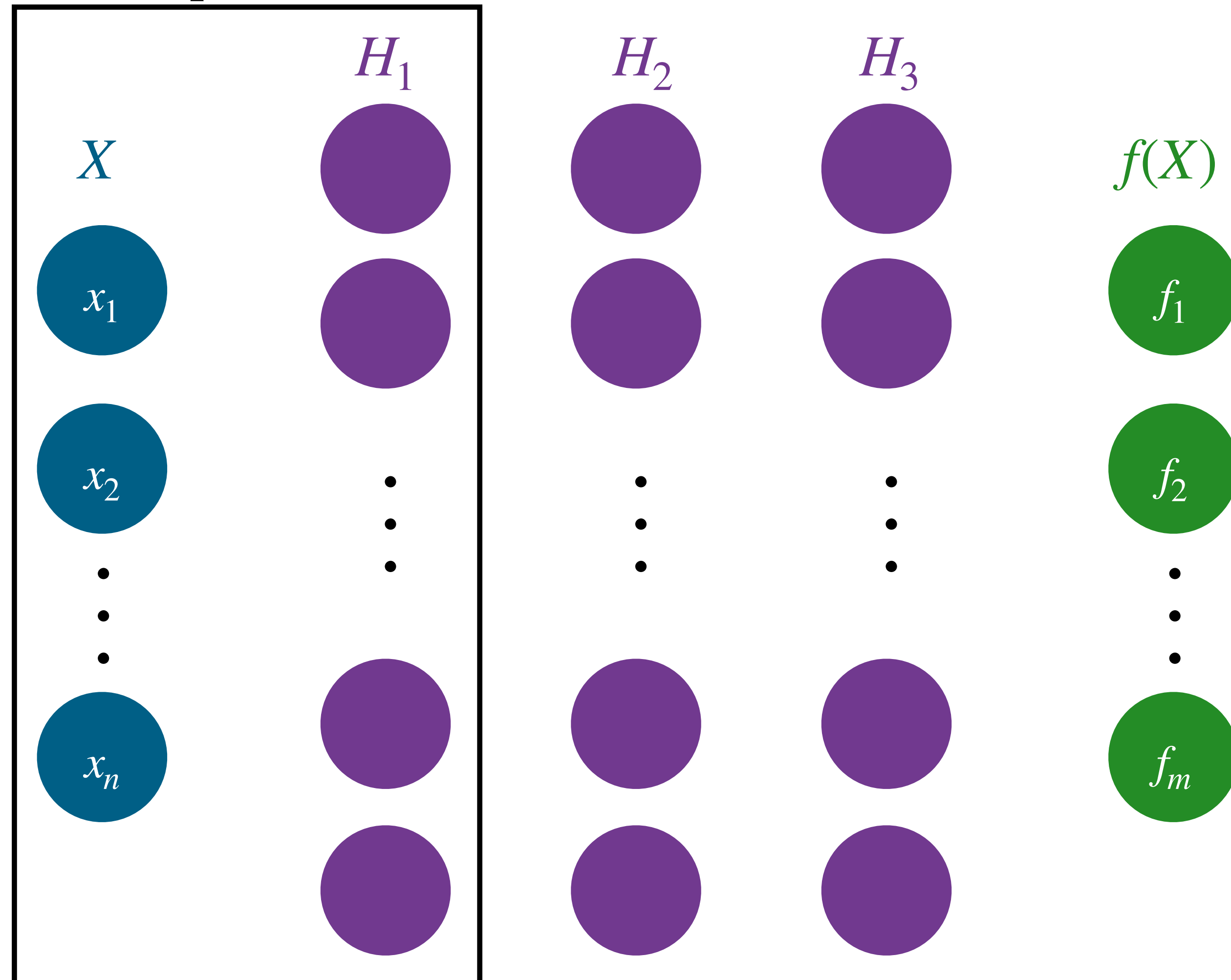
*guarantees a good approximation exists*

*but we don't know how many units you need, whether we can find the weights, or how the neural network behaves on data outside training*

# multi-layer perceptron

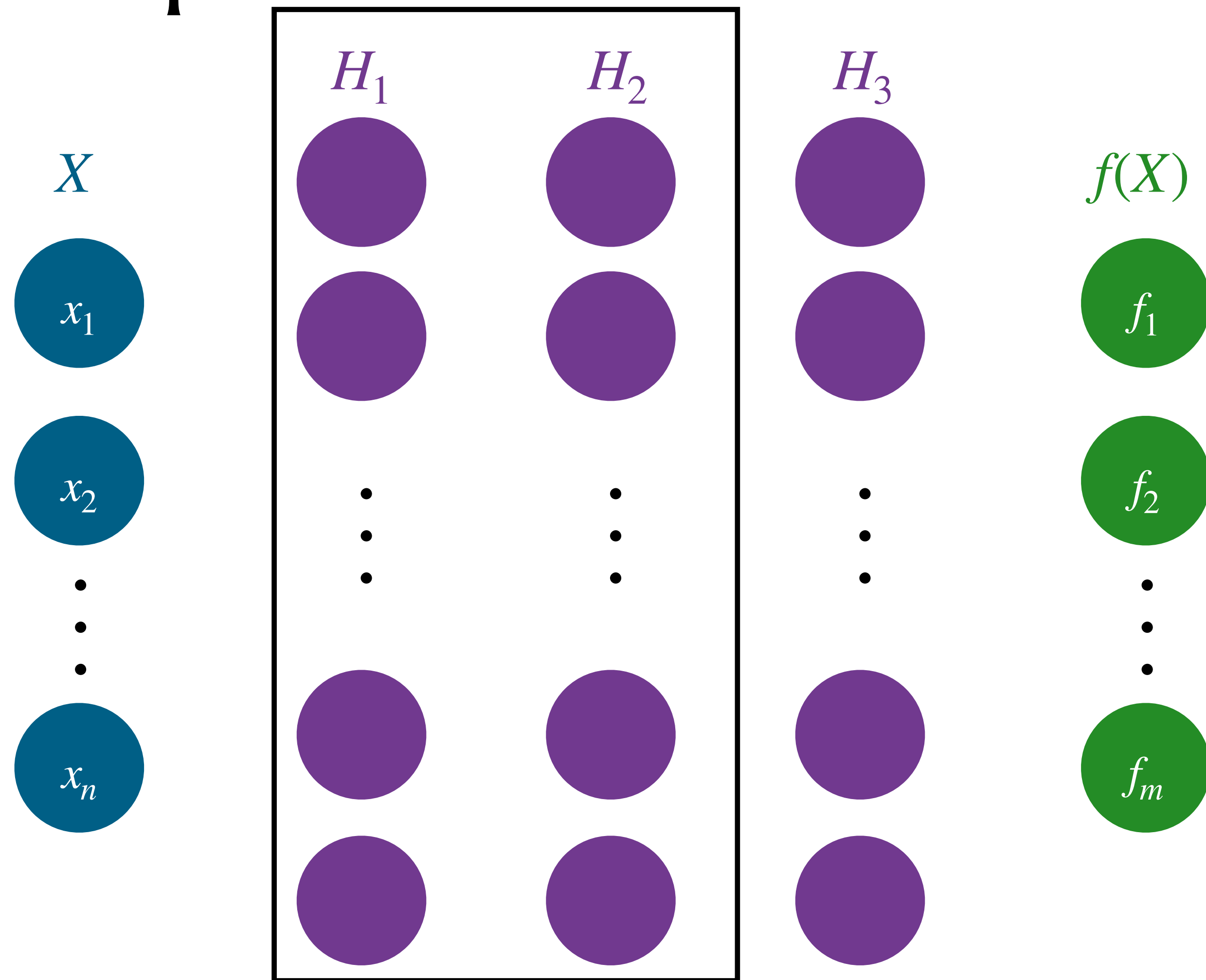


# multi-layer perceptron



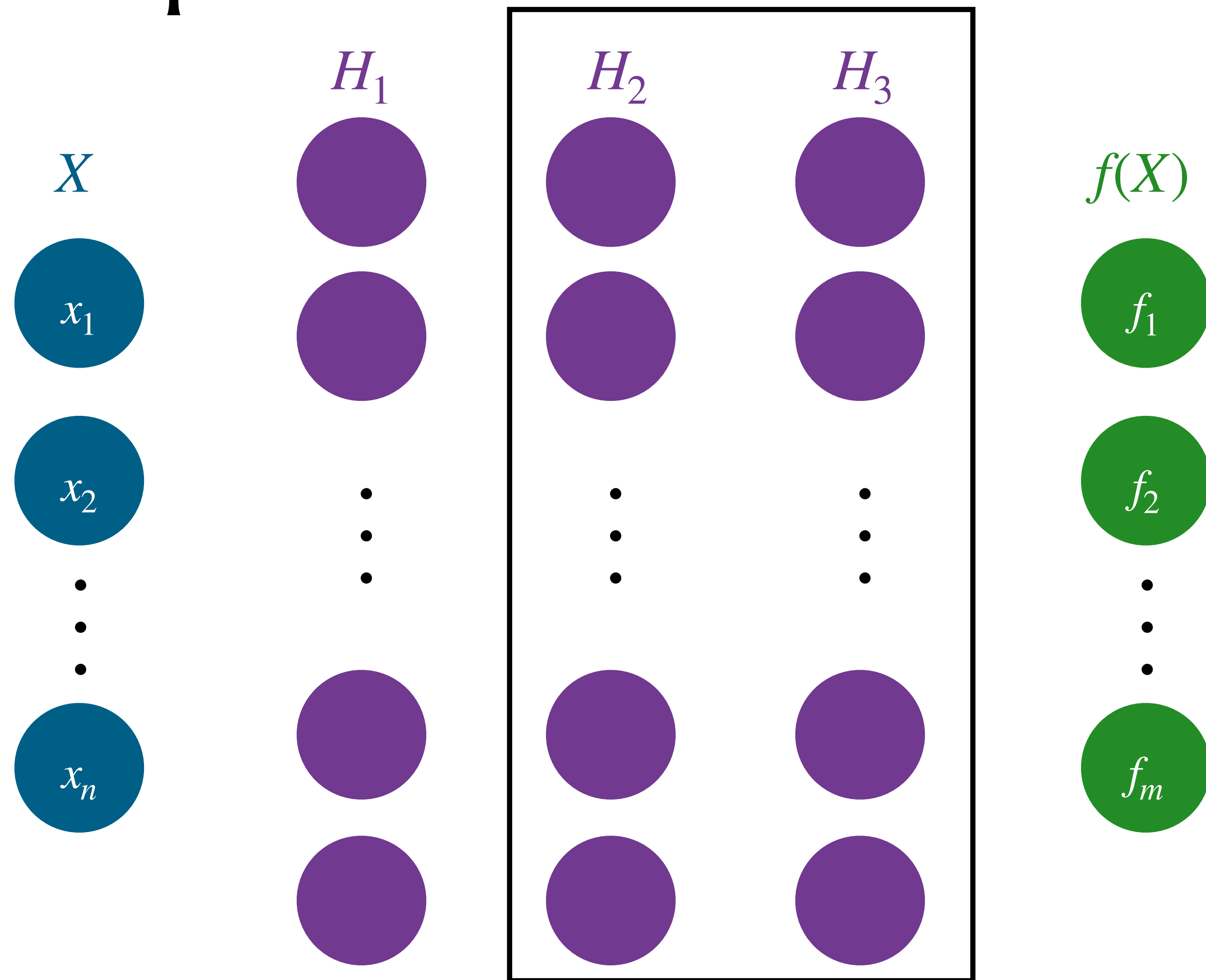
$$H_1 = \sigma(XW_1 + b_1)$$

# multi-layer perceptron



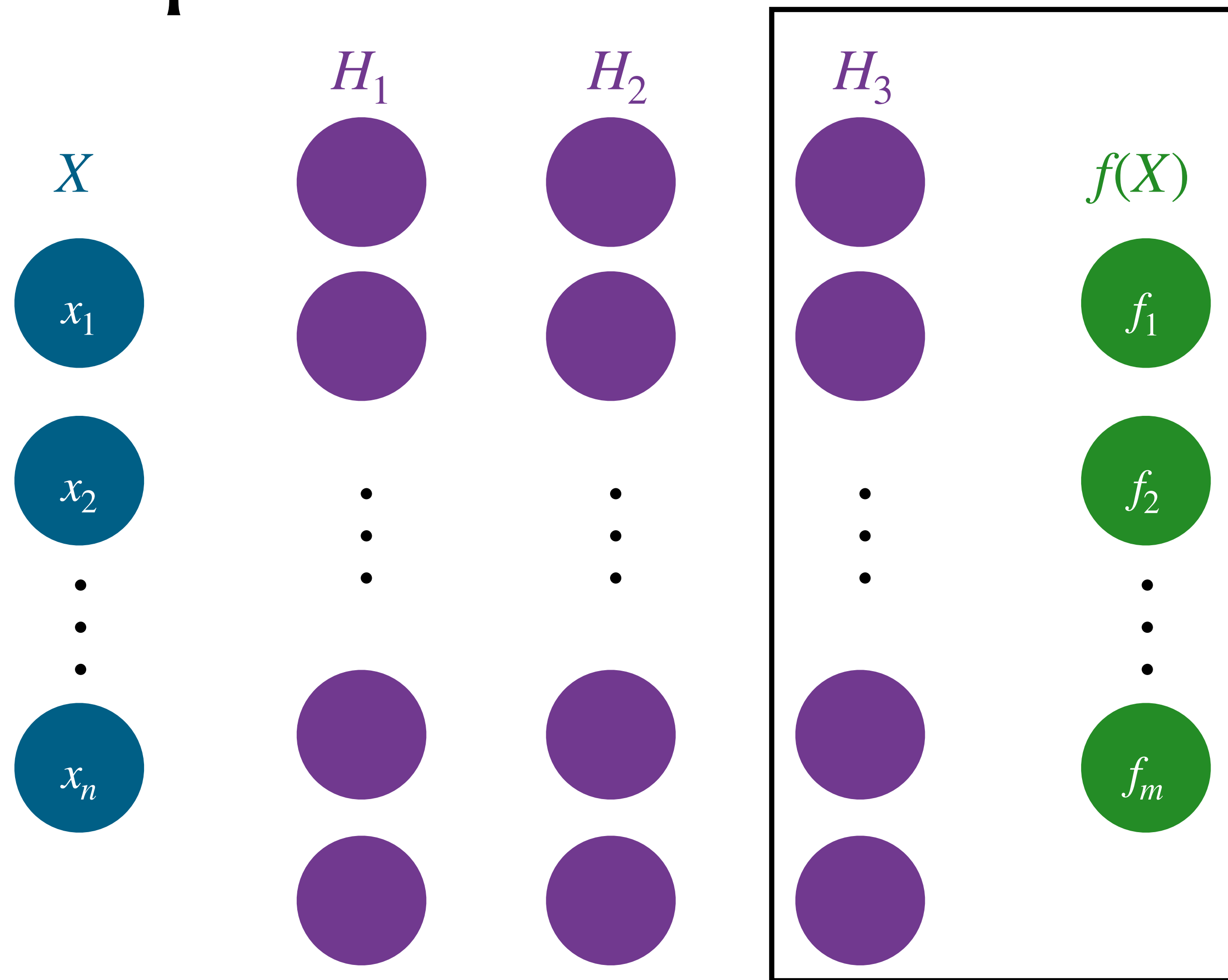
$$H_2 = \sigma(H_1 W_2 + b_2)$$

# multi-layer perceptron



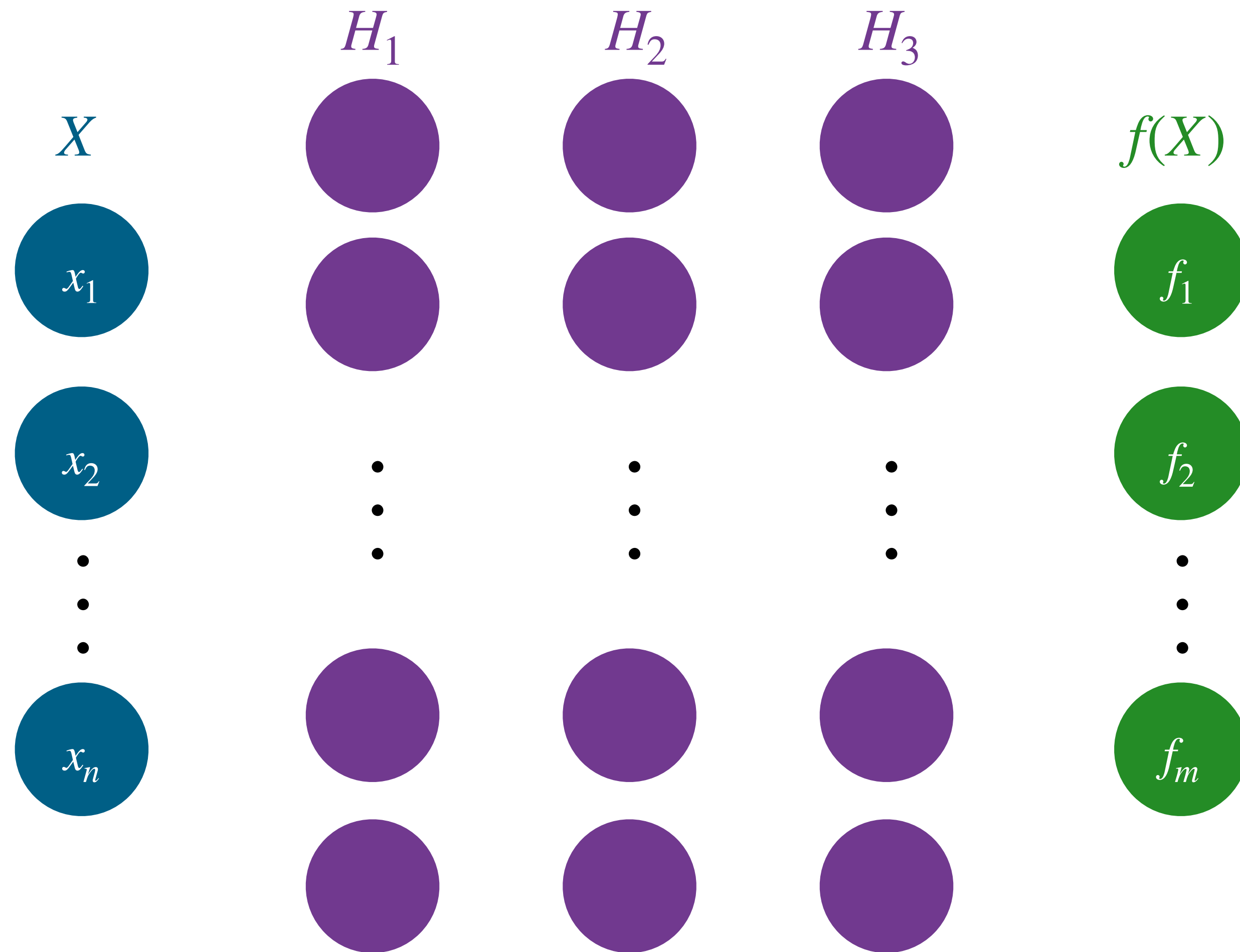
$$H_3 = \sigma(H_2 W_3 + b_3)$$

# multi-layer perceptron



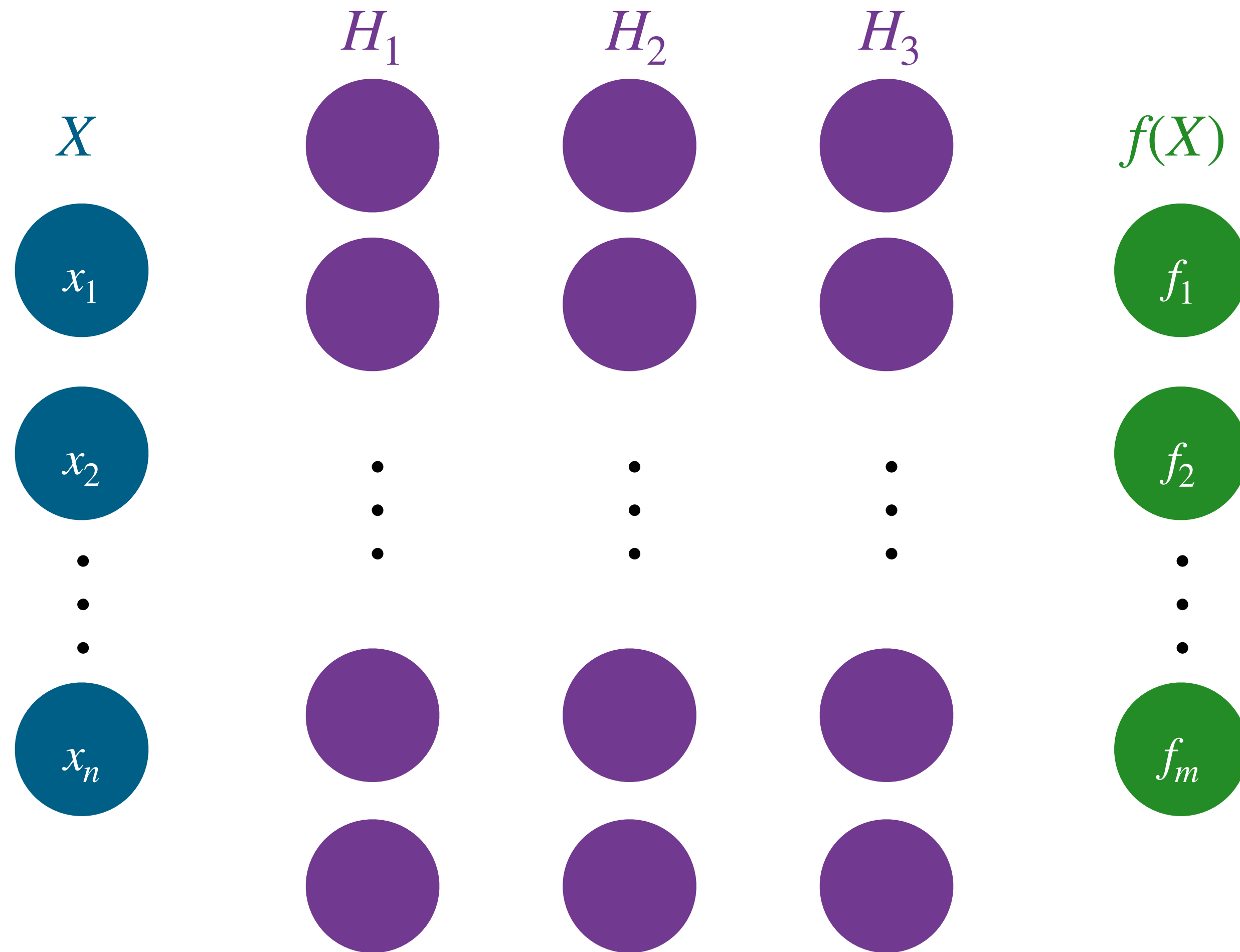
$$Y = H_3 W_4 + b_4$$

# multi-layer perceptron



for simplicity we'll refer to the MLP as  $f(x; w)$

# multi-layer perceptron



neural networks are just matrix multiplication — **GPU**

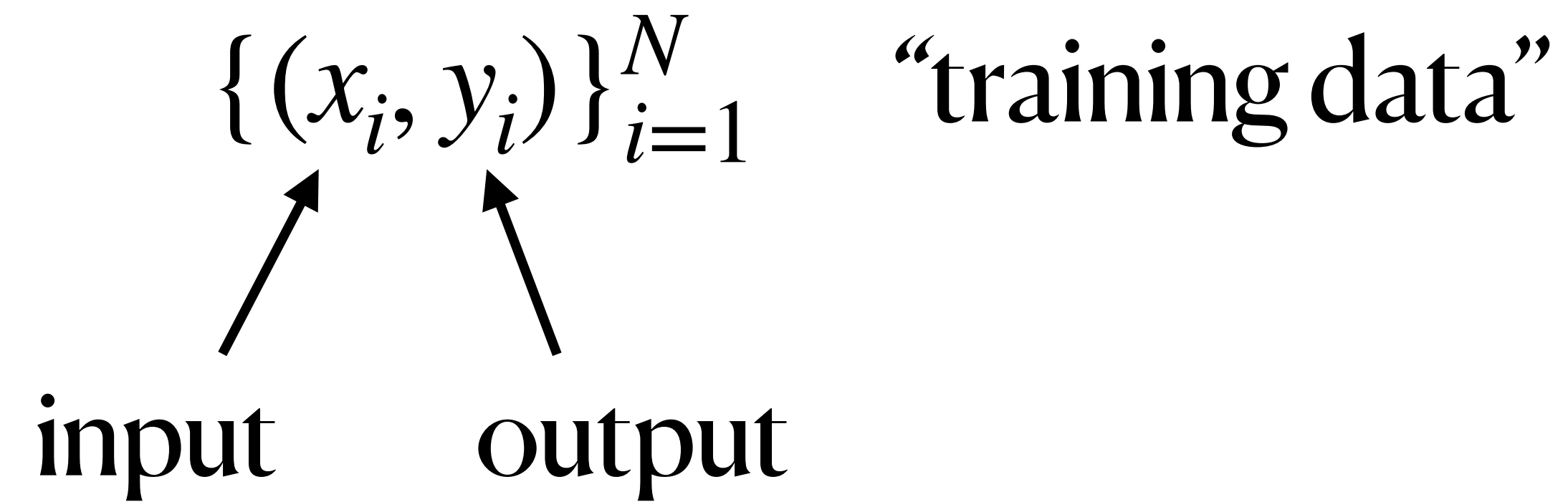
how do we learn the basis from data?

how do we learn the basis from data?

how do we learn the weights and biases from data?

how do we learn the basis from data?

how do we learn the weights and biases from data?



how do we learn the basis from data?

how do we learn the weights and biases from data?

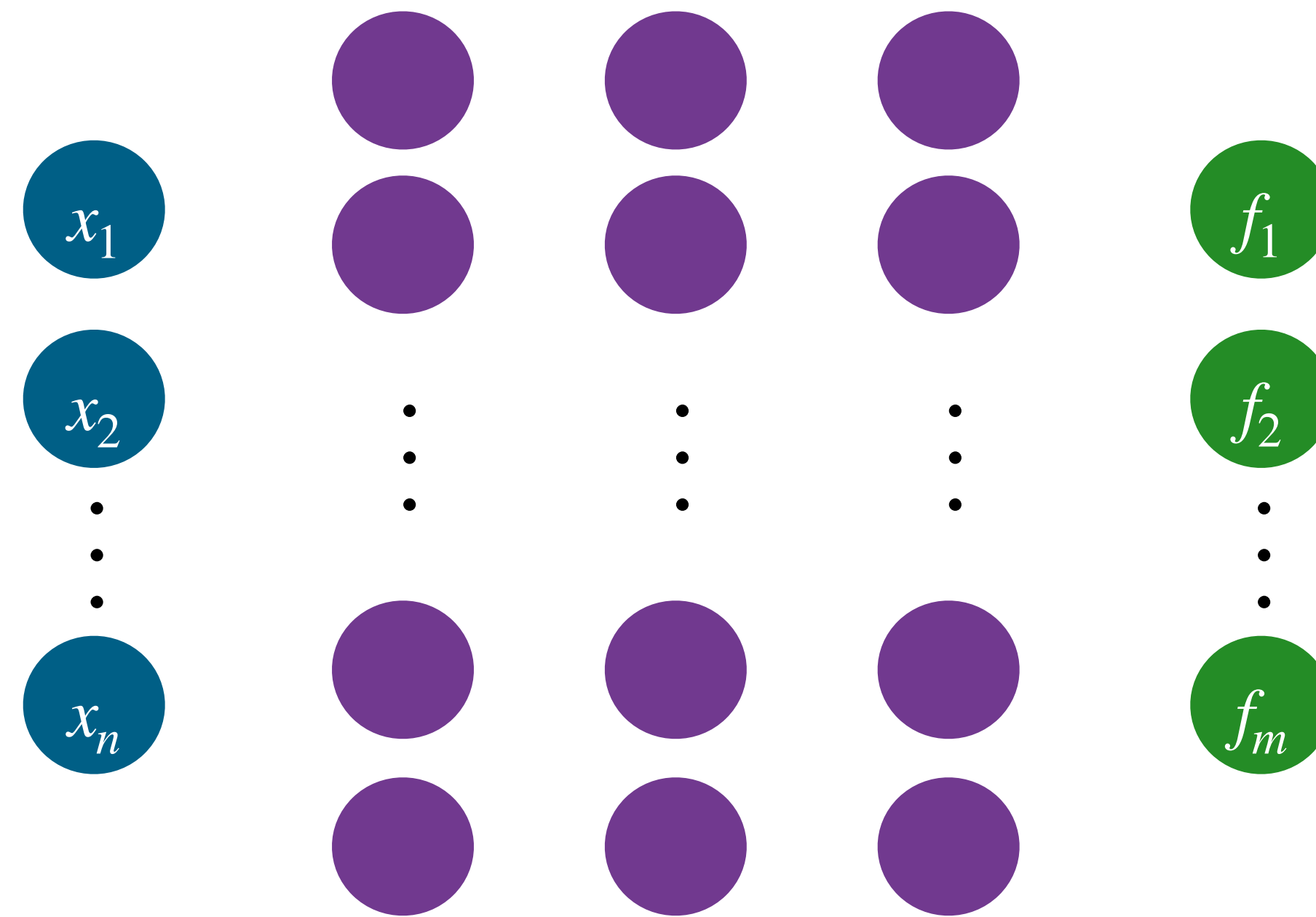
$$\{(x_i, y_i)\}_{i=1}^N \quad \text{“training data”}$$

$$\text{if } y_i = f(x_i; \mathbf{w}) + \varepsilon \text{ where } \varepsilon \sim \mathcal{N}(0, \sigma)$$

$$\text{then likelihood would be: } \log L(w) = - \sum_i \frac{(y_i - f(x_i; w))^2}{2\sigma^2} + \text{const}$$

full Bayesian inference with neural networks?

# full Bayesian inference with neural networks?



dimensions: *e.g.*

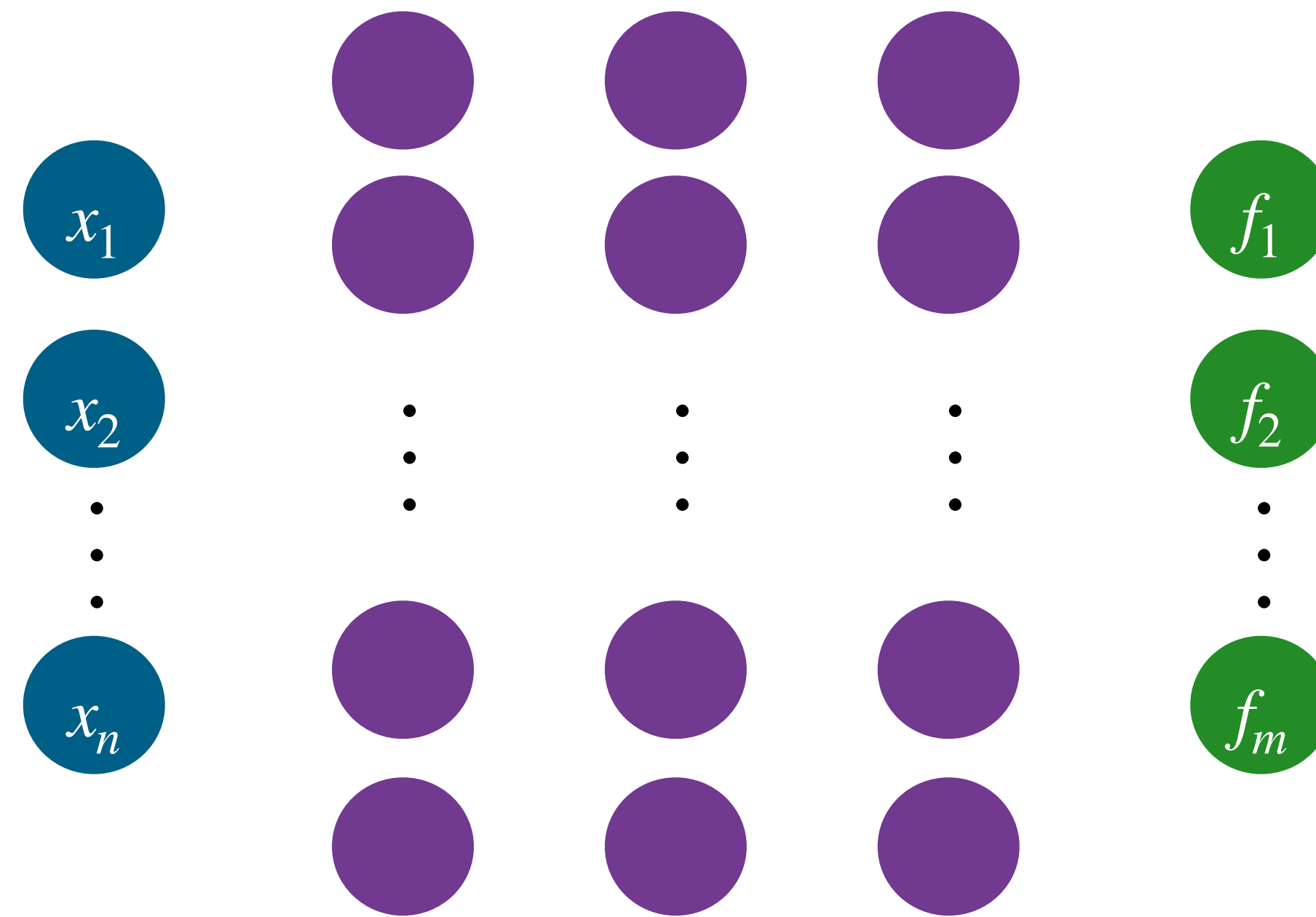
5      64      64      64      3

free parameter  
dimensions

$W_i : [5 \times 64] [64 \times 64] [64 \times 64] [64 \times 3]$   
 $b_i : 64 \qquad 64 \qquad 64 \qquad 3$   
384      4,160      4,160      192

tiny MLP has  
8,896 parameters

# full Bayesian inference with neural networks?



dimensions: *e.g.*

5      64      64      64      3

modern

free parameter  
dimensions

$W_i : [5 \times 64] [64 \times 64] [64 \times 64] [64 \times 3]$   
 $b_i : 64 \quad 64 \quad 64 \quad 3$   
 384      4,160      4,160      192

neural networks have  
 $10^8 - 10^9$

**optimizing** the likelihood/loss function —  
gradient descent and auto differentiation

neural networks and loss function can be written using JAX,  
PyTorch and other python packages with autodiff

*autodiff applies chain rule to elementary operations —  
neural networks are built in  $\times$  ,  $+$  , and activation function*

*see also lectures 5 and 8*

# optimizing the likelihood/loss function—backpropagation

consider a single layer  $a_i = h_{i-1}W_i + b_i$  and  $h_i = \sigma(a_i)$  let  $\delta_i = \frac{\partial L}{\partial a_i}$

# optimizing the likelihood/loss function—backpropagation

consider a single layer  $a_i = h_{i-1} W_i + b_i$  and  $h_i = \sigma(a_i)$  let  $\delta_i = \frac{\partial L}{\partial a_i}$

$$\frac{\partial L}{\partial W_i} = h_{i-1}^T \delta_i \quad \frac{\partial L}{\partial b_i} = \delta_i \quad \text{and} \quad \delta_{i-1} = (\delta_i W_i^T) \odot \sigma'(a_{i-1})$$

# optimizing the likelihood/loss function—backpropagation

consider a single layer  $a_i = h_{i-1}W_i + b_i$  and  $h_i = \sigma(a_i)$  let  $\delta_i = \frac{\partial L}{\partial a_i}$

$$\frac{\partial L}{\partial W_i} = h_{i-1}^T \delta_i \quad \frac{\partial L}{\partial b_i} = \delta_i \quad \text{and} \quad \delta_{i-1} = (\delta_i W_i^T) \odot \sigma'(a_{i-1})$$

as you run the network forward, keep  $a_i$  and  $h_i$ , then sweep backwards  
from the loss — gradient costs a fraction of a forward pass

# optimizing the likelihood/loss function—backpropagation

consider a single layer  $a_i = h_{i-1}W_i + b_i$  and  $h_i = \sigma(a_i)$  let  $\delta_i = \frac{\partial L}{\partial a_i}$

$$\frac{\partial L}{\partial W_i} = h_{i-1}^T \delta_i \quad \frac{\partial L}{\partial b_i} = \delta_i \quad \text{and} \quad \delta_{i-1} = (\delta_i W_i^T) \odot \sigma'(a_{i-1})$$

as you run the network forward, keep  $a_i$  and  $h_i$ , then sweep backwards  
from the loss — gradient costs a fraction of a forward pass

*No matter how many parameters: how we can train neural networks with billions of parameters*

# training a neural network in practice — **initialization**

standardize the inputs of the training data  $\{ (x_i, y_i) \}_{i=1}^N$  to zero mean and unit variance

if  $x_i$ s amplitude vary wildly, we'd get wildly different effective learning rates

# training a neural network in practice — **initialization**

standardize the inputs of the training data  $\{ (x_i, y_i) \}_{i=1}^N$  to zero mean and unit variance

initialize the weights  $\{ (W_i, b_i) \}$  at random

$W_i \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$  keeps every layer's output of order 1 (*He et al. 2015*)

# training a neural network in practice — **optimization**

for each **epoch**: *one pass through the training set*

**stochastic gradient descent**

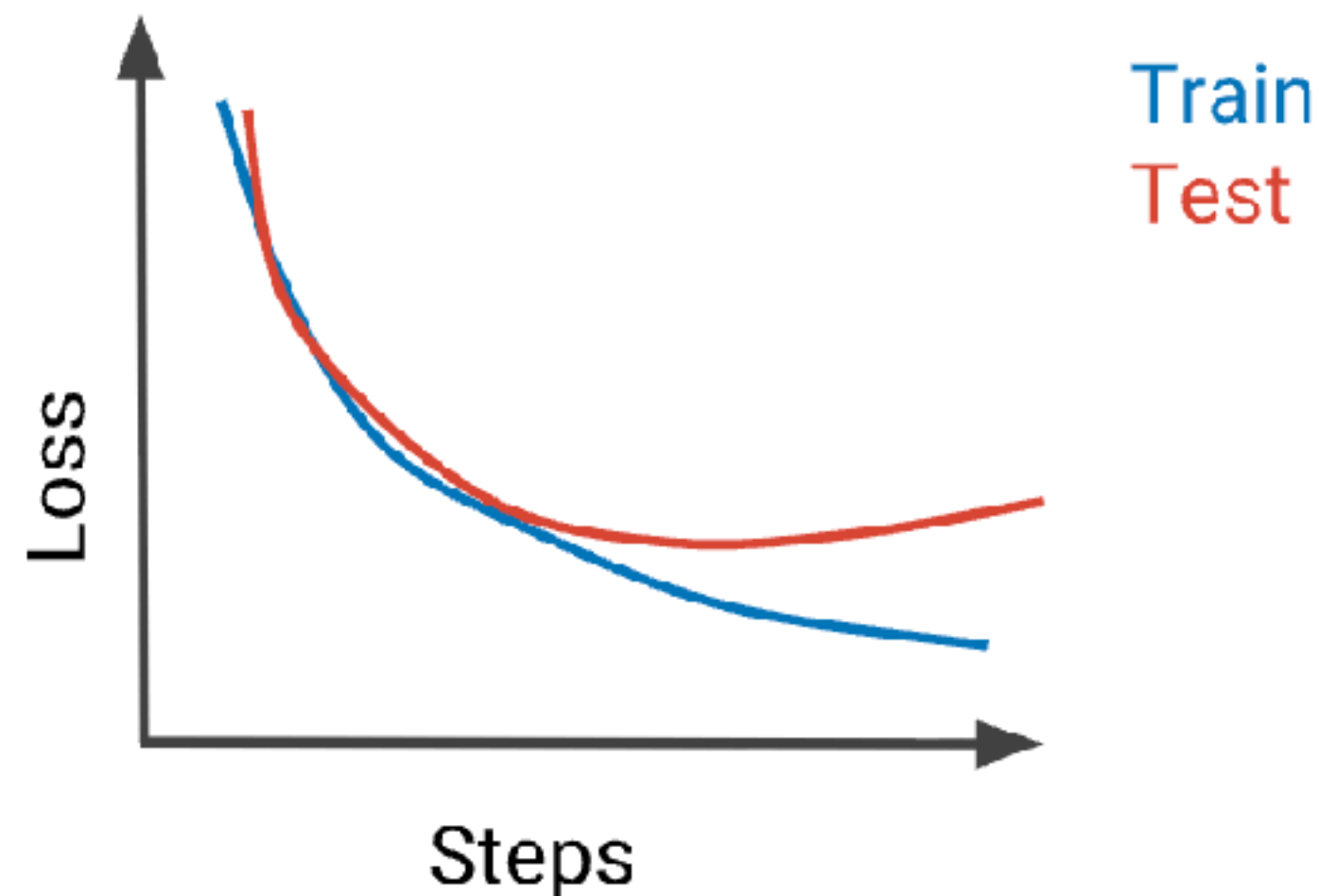
*gradients on random batches of the training data*

# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent

if you continue train indefinitely you will **overfit** the training data



# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent

if you continue train indefinitely you will **overfit** the training data

*with more parameters than data, you can a perfect fit to the noise*

# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent

**validation data** — held back data during training to evaluate model performance (not used for the gradient descent!)

# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent

**validation data** — held back data during training to evaluate model performance (not used for the gradient descent!)

80/10/10

# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent

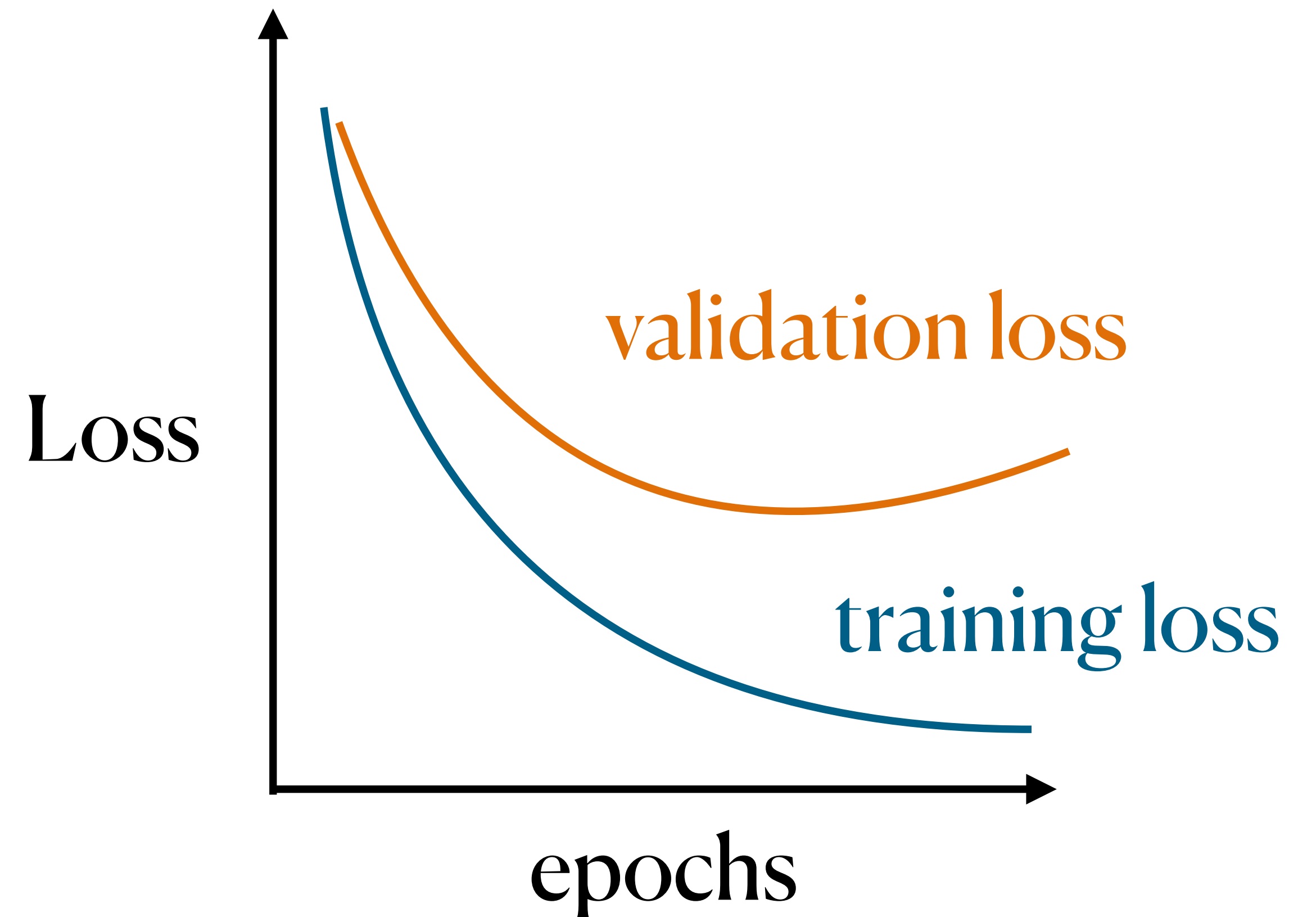
**validation data** — held back data during training to evaluate model performance (not used for the gradient descent!)

**early stopping** — stop training when it starts to overfit (*i.e.* when validation loss stops improving)

# training a neural network in practice — **generalization**

for each epoch:

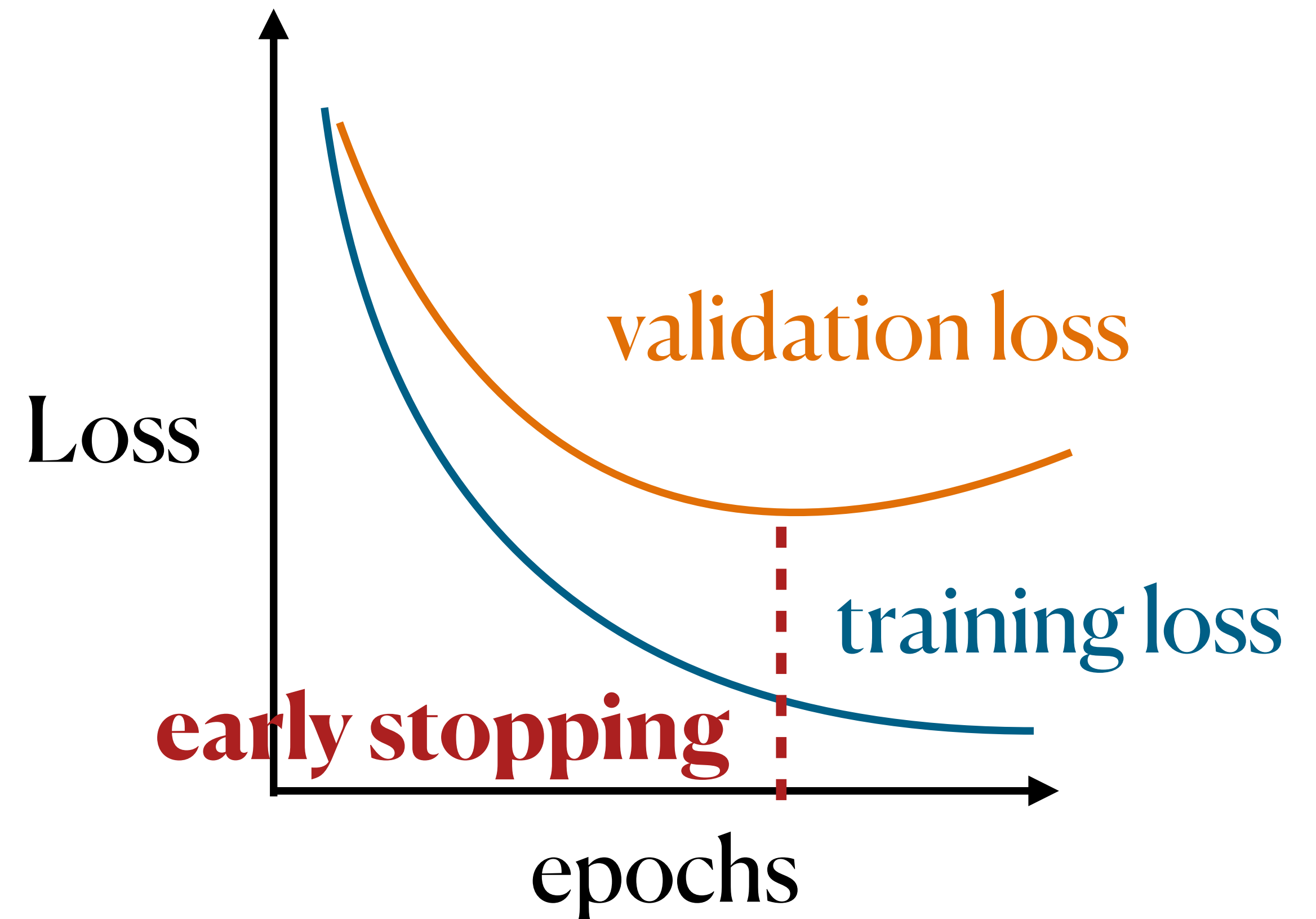
stochastic gradient descent



# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent



# training a neural network in practice — **generalization**

for each epoch:

stochastic gradient descent

**calculate validation loss**

*no gradient descent!*

**check for early stopping**

*e.g. stop training if validation loss does improve for 10 epochs*

training a neural network using `pytorch`

# training a neural network using pytorch

```
import pip install torch
```

```
# check: version
```

```
python -c "import torch as t;
```

```
print(t.__version__,t.cuda.is_available(  
) ,t.backends.mps.is_available())"
```

# training a neural network using `pytorch`

```
import torch
```

```
mu = torch.tensor(0.1) numpy array that can be stored on GPU
```

```
nll = torch.sum((y - mu)**2) backprop  
nll.backward()
```

```
mu.grad exact gradients
```

# training a neural network using pytorch

```
for epoch in range(30):
```

```
    # SGD minibatches
```

```
    order = torch.randperm(n_train)
```

```
    for k in range(0, n_train, 256):
```

```
        j = order[k:k + 256]
```

*batch*

```
        loss = 0.5 * torch.mean((forward(params, x[j]) - y[j])**2)
```

```
        loss.backward()
```

```
    # update parameters
```

```
    with torch.no_grad():
```

```
        for p in params:
```

```
            p -= lr * p.grad
```

```
            p.grad.zero_()
```

*pytorch accumulates gradients*

# training a neural network using pytorch

```
net = torch.nn.Sequential(torch.nn.Linear(5, 64),  
                           torch.nn.ReLU(),  
                           torch.nn.Linear(64, 64),  
                           torch.nn.ReLU(),  
                           torch.nn.Linear(64, 1))  
  
opt = torch.optim.SGD(net.parameters(), lr=0.05)  
  
for epoch in range(30):  
    for j in minibatches():  
        loss = 0.5 * torch.mean((net(x[j]) - y[j])**2)  
        opt.zero_grad()  
        loss.backward()  
        opt.step()
```

# training a neural network using pytorch

```
net = torch.nn.Sequential(torch.nn.Linear(5, 64),  
                           torch.nn.ReLU(),  
                           torch.nn.Linear(64, 64),  
                           torch.nn.ReLU(),  
                           torch.nn.Linear(64, 1))
```

holds a weight matrix  
and a bias

```
opt = torch.optim.SGD(net.parameters(), lr=0.05)
```

```
for epoch in range(30):  
    for j in minibatches():  
        loss = 0.5 * torch.mean((net(x[j]) - y[j])**2)  
        opt.zero_grad()  
        loss.backward()  
        opt.step()
```

# training a neural network using pytorch

```
net = torch.nn.Sequential(torch.nn.Linear(5, 64),  
                           torch.nn.ReLU(),  
                           torch.nn.Linear(64, 64),  
                           torch.nn.ReLU(),  
                           torch.nn.Linear(64, 1))  
opt = torch.optim.SGD(net.parameters(), lr=0.05)
```

holds a weight matrix  
and a bias

```
for epoch in range(30):  
    for j in minibatches():  
        loss = 0.5 * torch.mean((net(x[j]) - y[j])**2)  
        opt.zero_grad()  
        loss.backward()  
        opt.step()
```

optimizer to update the parameters

# **new groups!**

**Group 1: Ines, Jayden, Asa**

**Group 2: Madeline, Alexander, Daniel**

**Group 3: Nikhil, Noah, Michelle**

**Group 4: Tae Bong, Nicholas, Olivia**

**Group 5: Jules, Bella, Bocheng**

**Group 6: Kaleo, Shreya, John**

**in-class: photometric redshifts using an MLP**

**ast384c/inclass/lec12\_nn/photoz\_mlp.ipynb**